CARPETA DE CAMPO: PROYECTO INTERDISCIPLINARIO

GRUPO 6:

EMPRESA LOGÍSTICA

Santiago Agustín Pedraza Clérici

Escuela Técnica N° 32 D.E. 14 “Gral. José de San Martín

Computación

Profesores: Nahuel Acosta, Gabriel Abello Tuinyla

2026

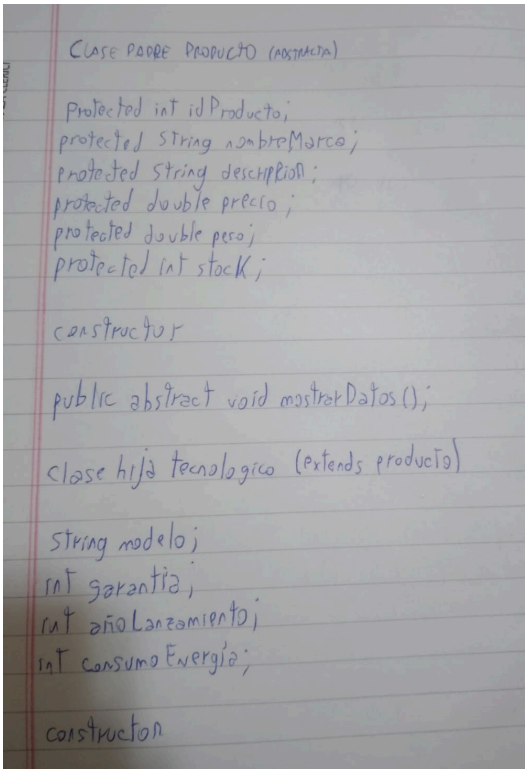
24/6: Lo primero que decidimos realizar fue una búsqueda de personas relacionadas al sector para poder entrevistarlas. Cada uno buscó por su parte entre sus personas cercanas, pero por mi parte no conseguí a nadie.

En la siguiente semana, rondando el **6/7**, nos juntamos en clase para proponer las clases y atributos que formarían parte del código en JAVA perteneciente al sistema.
Cada uno aportó con ideas. Yo propuse las clases hijas de la clase padre ‘’Productos”, las cuales serían si el producto es de tipo “tecnológico”, “decoración”, “indumentaria” o “accesorios”.

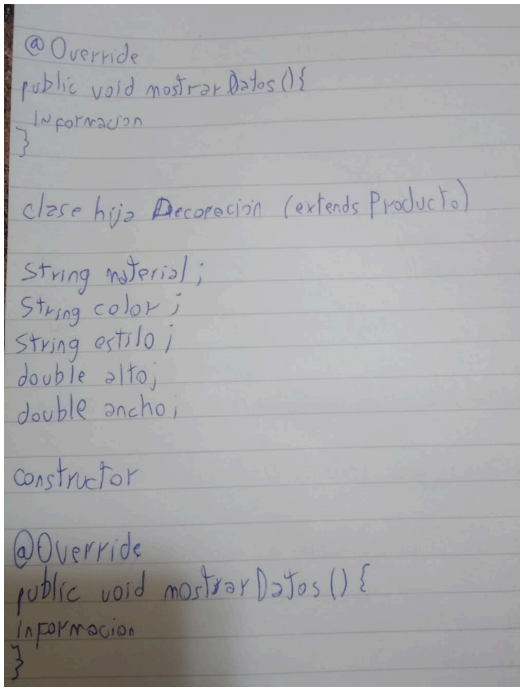
En el transcurso de esa semana, se nos asignó a cada uno realizar los códigos de las clases que cada uno propuso, como bocetos en papel. Se nos asignó fecha límite el domingo 12/7.

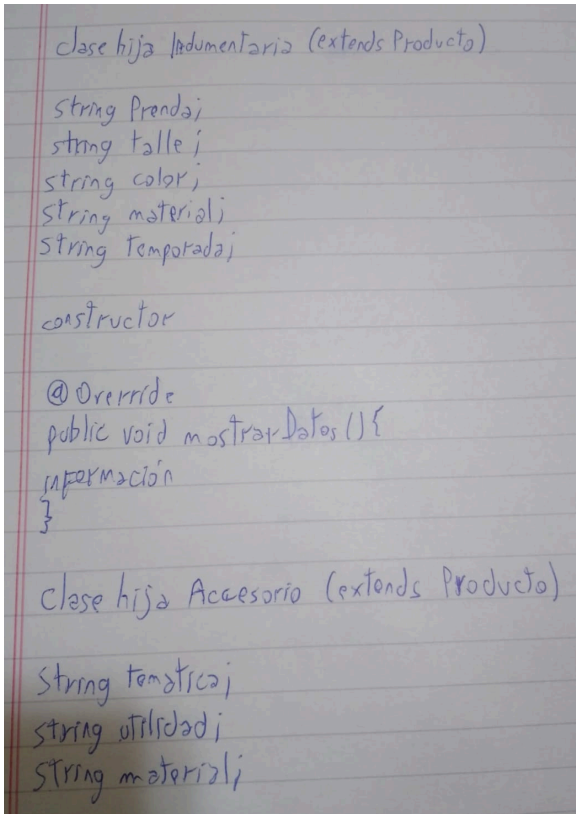
Por lo tanto, ese mismo dia de la fecha límite **12/7** (muy sobre el límite, hubiese preferido realizarlos antes pero no pude debido a que a la vez estudiaba para otros exámenes), realicé los bocetos en papel requeridos, en mi caso, los correspondientes a la clase ‘’Productos’’ y sus clases hijas “tecnológico”, “decoración”, “indumentaria” o “accesorios” , ya que fueron los que plantee yo el día que propusimos todas las clases que formarán parte del sistema.
Estos códigos, a futuro, los pasaremos en limpio a la computadora en NetBeans.

Clase Padre “Producto”:

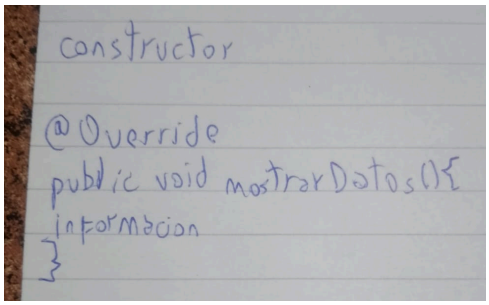


Parte de la clase padre Producto y clase hija Decoración:





Clase hija “Indumentaria” y parte de la clase hija “Accesorio”.

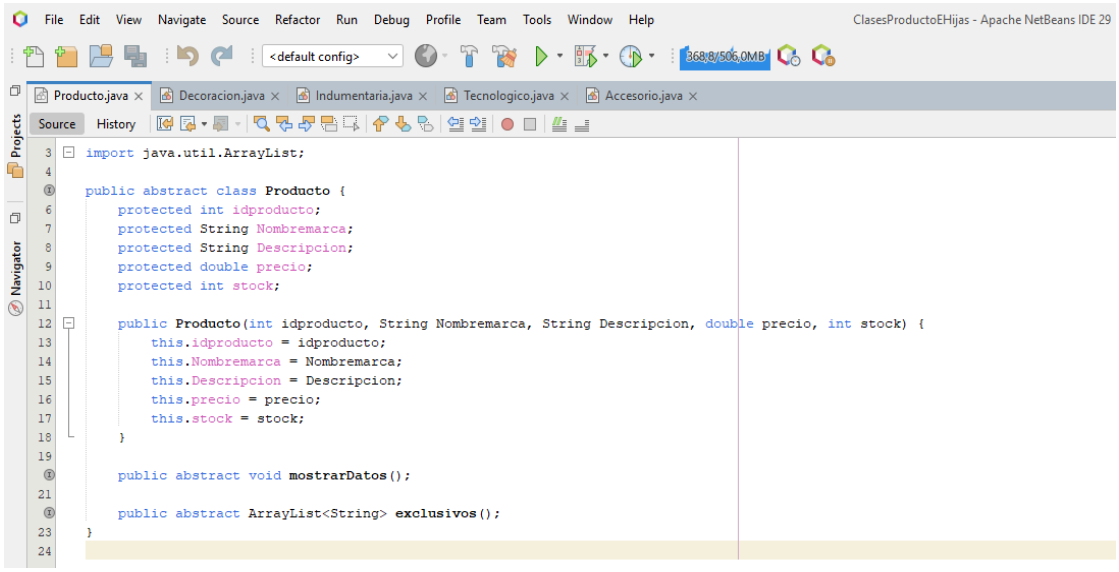


Final de la clase hija “Accesorio”.

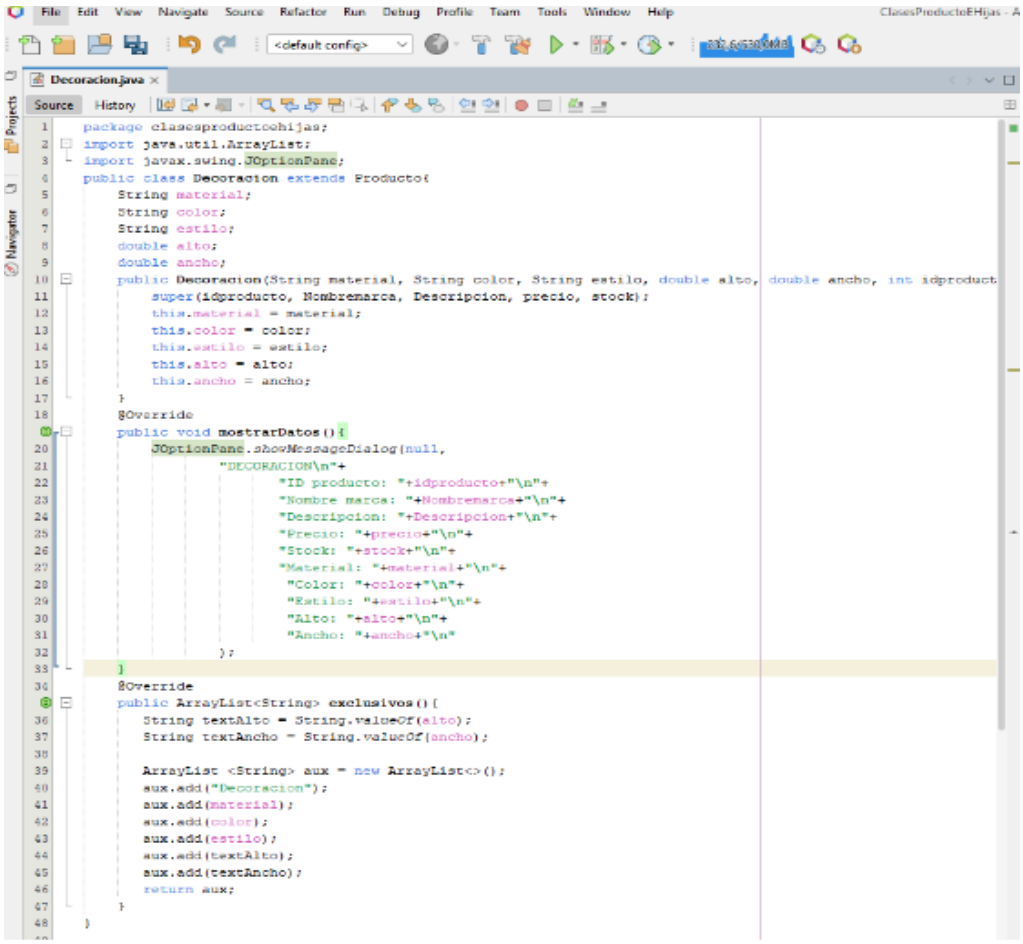
Una vez que todos los integrantes realizaron sus bocetos y que fueron todos corroborados y corregidos por el lider Santiago Goldfarb y el compañero Benjamin Saragusti, se nos dio finalmente la orden de pasar en limpio los bocetos.

Por lo tanto, el dia 15/7 cumpli con mi deber, y plasme tal cual mis bocetos de la clase padre “Producto” e hijas (ya que en mi caso, no contenían errores) en la computadora a NetBeans.

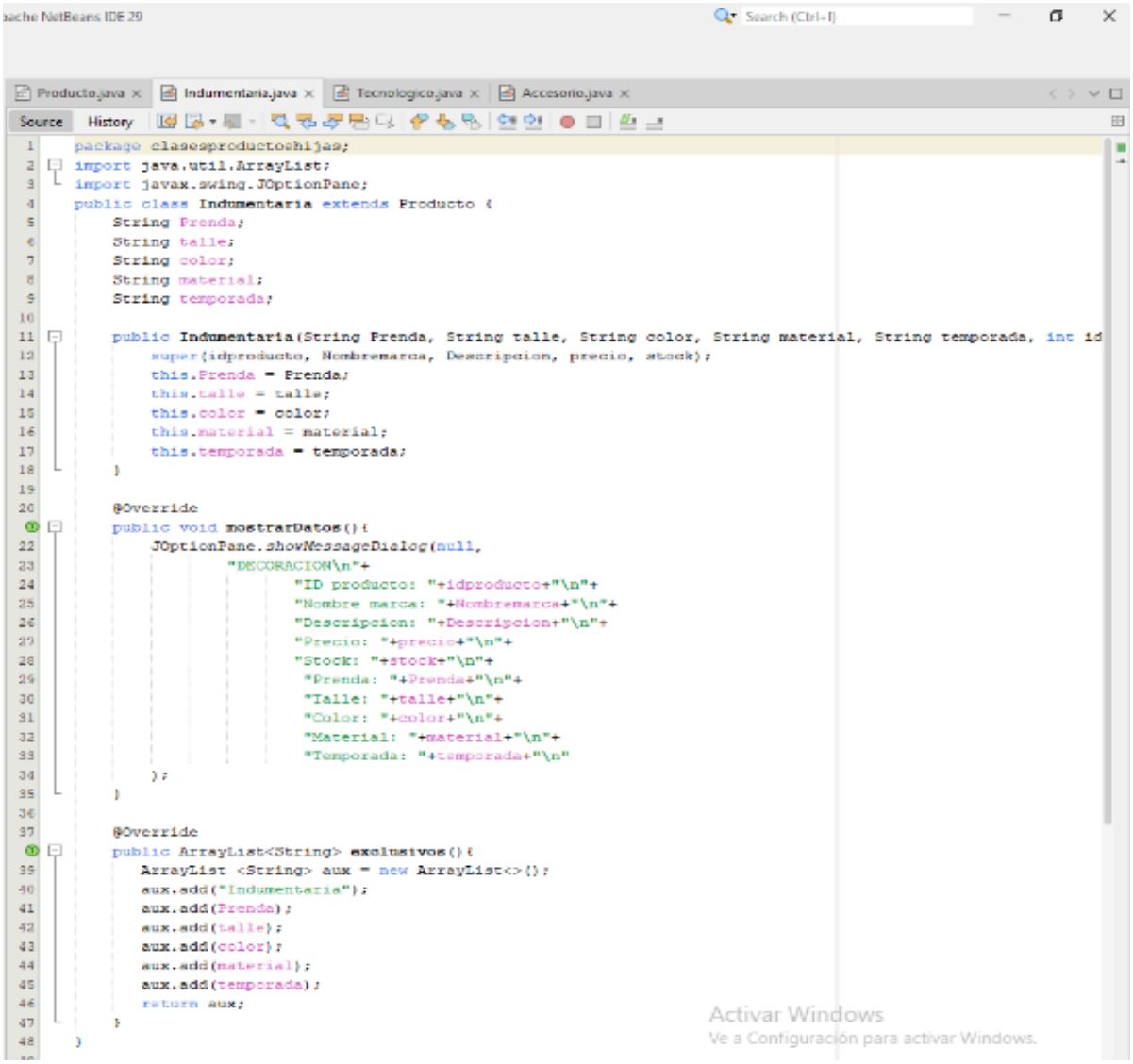
Clase Padre Producto:



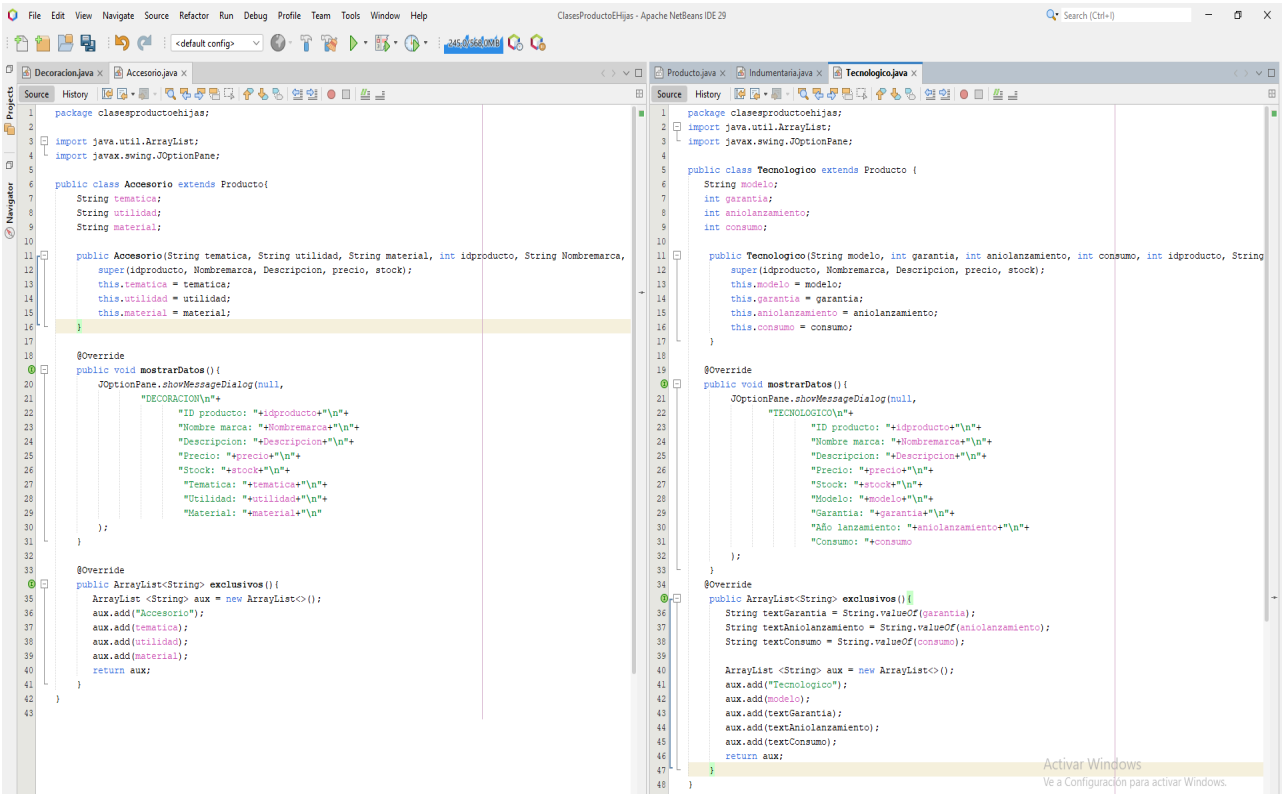
Clase Hija “Decoración”:



Clase hija “Indumentaria”:



Clases hijas “Accesorio” y “Tecnológico”



```
package clasesproductoehijas;

import java.util.ArrayList;
import javax.swing.JOptionPane;

public class Accesorio extends Producto{
    String tematica;
    String utilidad;
    String material;

    public Accesorio(String tematica, String utilidad, String material, int idproducto, String Nombremarca,
        super(idproducto, Nombremarca, Descripcion, precio, stock);
        this.tematica = tematica;
        this.utilidad = utilidad;
        this.material = material;
    }

    @Override
    public void mostrarDatos(){
        JOptionPane.showMessageDialog(null,
                "DECORACION\n"+
                        "ID producto: "+idproducto+"\n"+
                        "Nombre marca: "+Nombremarca+"\n"+
                        "Descripcion: "+Descripcion+"\n"+
                        "Precio: "+precio+"\n"+
                        "Stock: "+stock+"\n"+
                         "Tematica: "+tematica+"\n"+
                         "Utilidad: "+utilidad+"\n"+
                         "Material: "+material+"\n"
        );
    }

    @Override
    public ArrayList<String> exclusivos(){
       ArrayList <String> aux = new ArrayList<>();
       aux.add("Accesorio");
       aux.add(tematica);
       aux.add(utilidad);
       aux.add(material);
       return aux;
    }
}
```

```
package clasesproductoehijas;
import java.util.ArrayList;
import javax.swing.JOptionPane;

public class Tecnologico extends Producto {
   String modelo;
   int garantia;
   int aniolanzamiento;
   int consumo;

    public Tecnologico(String modelo, int garantia, int aniolanzamiento, int consumo, int idproducto, String
        super(idproducto, Nombremarca, Descripcion, precio, stock);
        this.modelo = modelo;
        this.garantia = garantia;
        this.aniolanzamiento = aniolanzamiento;
        this.consumo = consumo;
    }

    @Override
    public void mostrarDatos(){
        JOptionPane.showMessageDialog(null,
                "TECNOLOGICO\n"+
                        "ID producto: "+idproducto+"\n"+
                        "Nombre marca: "+Nombremarca+"\n"+
                        "Descripcion: "+Descripcion+"\n"+
                        "Precio: "+precio+"\n"+
                        "Stock: "+stock+"\n"+
                        "Modelo: "+modelo+"\n"+
                        "Garantia: "+garantia+"\n"+
                        "Año lanzamiento: "+aniolanzamiento+"\n"+
                        "Consumo: "+consumo
        );
    }
    @Override
    public ArrayList<String> exclusivos(){
        String textGarantia = String.valueOf(garantia);
        String textAniolanzamiento = String.valueOf(aniolanzamiento);
        String textConsumo = String.valueOf(consumo);

        ArrayList <String> aux = new ArrayList<>();
        aux.add("Tecnologico");
        aux.add(modelo);
        aux.add(textGarantia);
        aux.add(textAniolanzamiento);
        aux.add(textConsumo);
        return aux;
    }
}
```

Si bien los códigos de las clases los realice en el laboratorio del colegio, ya que fue un dia miércoles, las capturas son sacadas desde mi computadora (se puede ver en el icono de “Activar Windows”), esto debido a que me los pase por mail a mi mismo, para evitar una posible pérdida de los mismos, en caso de que la PC del colegio sufra un congelamiento.

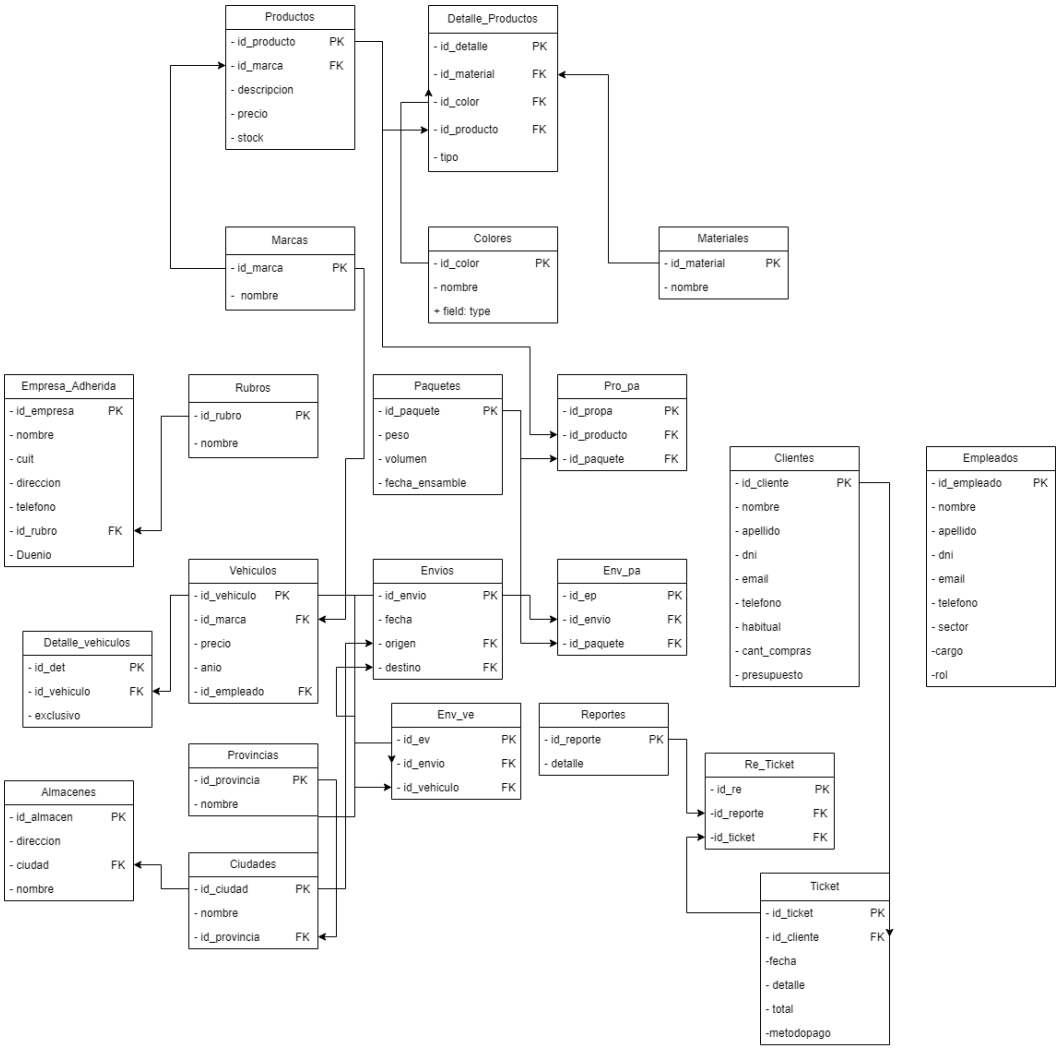
20/7 - 31/7 VACACIONES DE INVIERNO

Finalizadas las vacaciones de invierno, el día miércoles **5/8** tuvimos el primer “checkpoint” en el cual los profesores Nahuel Acosta y Gabriel Tuinyla corroboraron todo lo realizado por el grupo hasta el momento. Una vez finalizada la corrección (en la cual, exceptuando algunos detalles,

nos fue bastante bien), mi siguiente tarea fue realizar la primera versión del DER (diagrama entidad-relación) para basarse en él al realizar el futuro código de la base de datos en MySQL Workbench 8.0. Por lo tanto, ese mismo día, realice completo el DER, basándose en unas anotaciones en papel que me proporcionó mi compañero Benjamin Saragusti, las cuales contenían los nombres y algunos atributos de las tablas normales y de unión.

El DER incluye todas las tablas y atributos que, hasta este momento, forman parte del sistema.

Primera versión del DER:



Posteriormente, Benjamin Saragusti modificó un par de detalles mínimos.

Al día siguiente, el **6/8,** participamos todos en la búsqueda de los elementos que formarán parte de la infraestructura de la empresa, incluyendo PCs, periféricos, routers, switches, cables, etc., para posteriormente calcular el presupuesto final. Intentamos emplear la menor cantidad de dinero posible.

Yo busqué y proporcioné los links de todos los componentes y elementos (la mayoría de ellos, sacados de “Mercado Libre”). Los componentes que encontre yo y se terminaron implementando al listado final son las computadoras con todos sus componentes, el switch gigabit de 24 puertos, el router cisco 2811 con 2 puertos EtherNet, cable de red vention 2 metros 40 gbps, el cable coaxil fino, el teléfono IP, la impresora red laser, el plotter de corte 63 cm láser y la cámara de seguridad (Imou Ranger Se Wifi Audio 360° 2mp Alarm).

Como presupuesto final nos dio: $695841416.

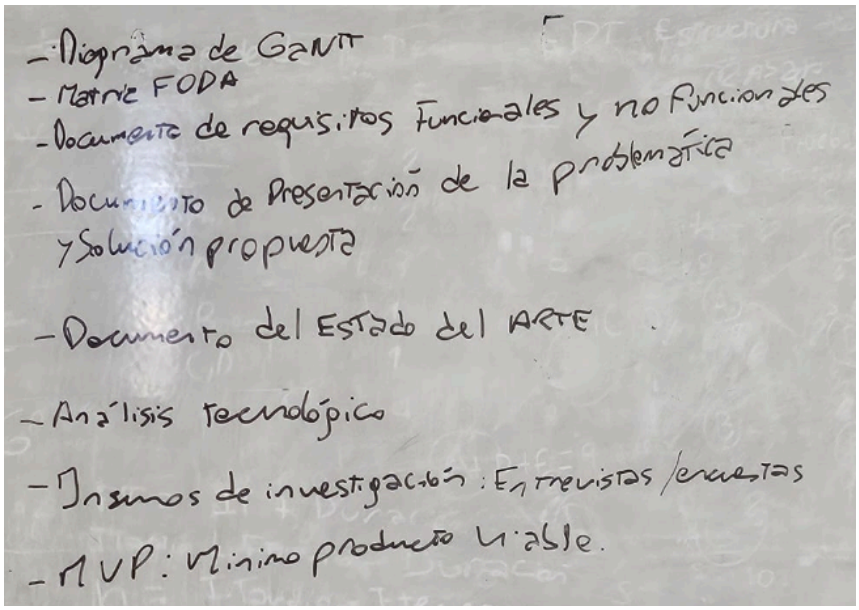
Realizado en hoja de cálculo de google.

1	Presupuesto Compañia de logistica				
2	Computadoras :	Unidad	Cantidad	Precio total	Total de todo
3	Procesador:Intel core I5 4Gen				13121266
4	Ram: 16gb DDR3				
5	GPU:Intel graphics				
6	Disco duro: SSD 256gb				
7	Windows 10				
8	Puerto Ethernet				
9	Modelo HP PRO DESK				
10	Placa: No dice				
11	Monitor 19 pulgadas				
12	Mouse y teclado				
13	Precio	450000	26	11700000	
14	Switch				
15	Modelo Switch gigabit 24 puertos	169491	5	847455	
16	Router				
17	modelo multi wan gigabit 4 wan 5 puertos 1 lan	79999	3	239997	
18	Cables :				
19	Cable ethernet:				
20	Cable de red vention 2 metros 40gbps	11589	26	301314	
21	Cable coaxil fino				
22	5 metros rg6	6500	5	32500	

980 filas no modificadas

Presupuesto

El día **7/8** el profesor Gabriel Tuinyla nos encargó las siguientes consignas a todos los grupos con fecha límite 14/8.



Por lo tanto, ese mismo día, nos dividimos las consignas entre todos los miembros del grupo.
A mi se me asignó realizar la documentación de la presentación de la problemática y solución propuesta.

Entre los días **9/8** y **10/8** finalicé con eso.
Para su realización, utiliza como fuentes principales gran cantidad del material realizado por mis demás compañeros hasta entonces de la sección “Análisis de sistemas” (principalmente los análisis de requerimientos, el MVP Mínimo Producto Viable, el análisis FODA Fortalezas, Oportunidades, Debilidades y Amenazas, y la entrevista).
Aun así, utilice la inteligencia artificial (ChatGPT) para que me corrija errores de redacción o me aclare si faltaba contenido vital para una documentación de ese estilo.

El documento consta de cuatro páginas, el cual incluye:

- **Introducción y descripción de la problemática**:

  Se presenta el tipo de proyecto que se está realizando, quien lo utilizara, listado de las actividades que suele realizar a diario quien utilizara luego el sistema (las mismas actividades que el sistema le facilitará realizar).

- **Solución propuesta**:

  Se explican brevemente los procesos y funciones que el programa realizará, junto con sus ventajas.

- **Objetivos específicos**:

  Ofrece un listado de ítems en los cuales se mencionan doce de los principales objetivos los cuales el programa satisfacerá.

- **Funcionamiento general del sistema**:

  Se muestra un ejemplo textual de una operación logística, dividido en siete secciones, el cual se podrá realizar mucho más fácilmente mediante el programa, una vez finalizado el mismo.

- **Beneficios esperados:**

  Ofrece un listado de ítems en los cuales se mencionan, como dice el título, los principales beneficios que se esperan que el usuario gozará al utilizar el programa en cuestión.

Enlace al documento original:

Documento de presentación de la problemática y solución propuesta - Santiago Agustin Pe…

Lo realice en un documento de google, aunque la versión que se encuentra en el repositorio de GitHub está en formato PDF.

Durante la siguiente semana, mi compañero y líder Santiago Goldfarb comenzó a realizar un boceto de la que sería una posible versión de la página web del proyecto. Dicho boceto se realizó en la página “Figma”. Por lo tanto, alrededor del **17/8**, me sume a dicha actividad.

Como tal eran dos los bocetos, uno para el sitio web de los administradores y otro para el sitio web general.
Usamos como ‘’plantilla’’ o ‘’referencia’’ el sitio web de la empresa de envíos Andreani.
Por mi parte, realice tres distintas secciones correspondientes al sitio al cual solo podía acceder el administrador.

La primer seccion que diseñe, corresponde a la del Seguimiento del producto.
La idea de la misma es que, ademas de su header y footer correspondientes, muestre un mapa en el cual se señale, mediante un punto rojo de ubicacion, el sitio exacto y en tiempo real en el que se encuentra el/los producto/s que estan siendo enviados.
Ademas, un texto con un contador en el cual muestre el tiempo aproximado restante hasta que el paquete llegue a su destino, un texto que mencione en qué tipo de vehículo está siendo transportado el paquete y un texto que muestre la dirección del destino.

Diseño de la sección:



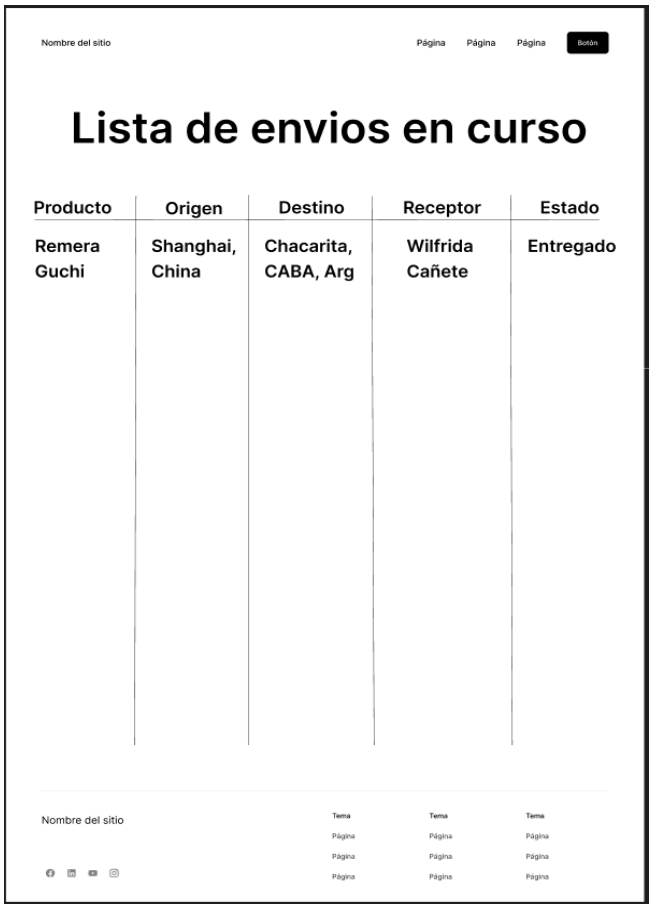
Al tratarse de un boceto y de no conocer con detalle todas las opciones y funcionalidades de ‘’Figma’’, el boceto tiene un formato como el de una hoja A4, y no como el de un sitio web real.

Para el mapa, busque una ubicación aleatoria en google maps (en este caso, el barrio de Monserrat), tome una captura de pantalla y la pegue en el boceto. Para el punto de

ubicación, la casita de la dirección, la flecha negra, el reloj para el contador y el camioncito para mostrar el tipo de vehículo los busque como imágenes sin derechos de autor en google, los descargue y los pegue (algunos tuve que quitarles manualmente el fondo en páginas como remove.bg).

La siguiente sección que realicé fue una que permite ver un listado en tiempo real de los envíos en curso, mostrando el nombre del producto, lugar de origen, lugar de destino, nombre del receptor y estado del envío.

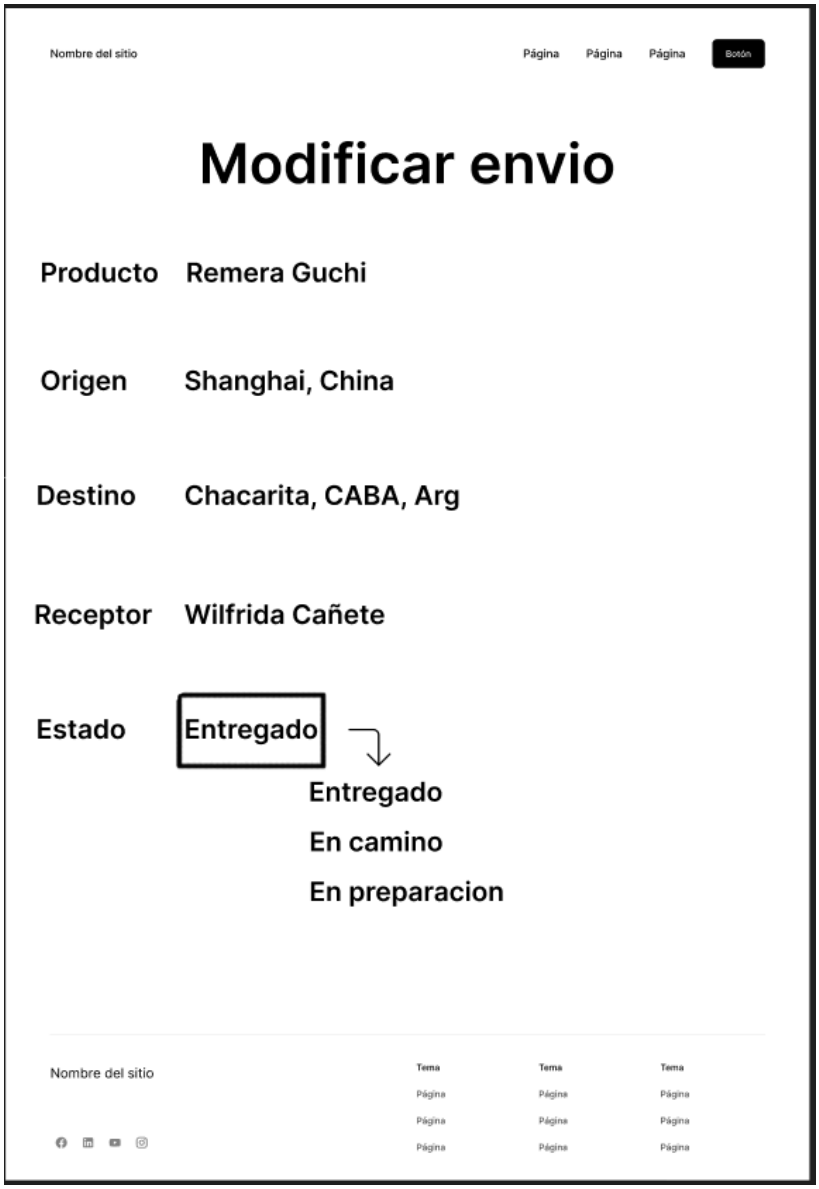
Para su diseño, no utilice imágenes externas, solo herramientas de Figma.



La última sección que diseñé fue la correspondiente a la opción de “Modificar envío”. A esta sección se accedería al hacer click sobre cualquier envío del listado de envíos en curso.

Posee un renglón para cada atributo de los mostrados en la lista, los cuales el administrador podrá modificar en caso de ser necesario. Para el atributo “estado”, al hacer click en él, desplegaría tres posibles opciones predeterminadas, las cuales serán “Entregado”, “En camino” y “En preparación”.

Tampoco utilice imágenes externas, solo herramientas de Figma.



27/8: Agustín Zowmir y yo recibimos la orden de realizar el menú interactivo en el main del código realizado en Java, utilizando JOptionPane como medio de ingreso de datos por parte del usuario.

El menú consiste en una lista de opciones, de las cuales el usuario puede seleccionar una ingresando el número correspondiente. Según el número ingresado, se lleva a cabo la función correspondiente mediante un switch-case.

Las opciones disponibles son:

1) Agregar cliente.
2) Agregar empleado.
3) Agregar reporte.
4) Agregar envío.
5) Agregar paquete.
6) Agregar producto.
7) Agregar vehículo.
8) Agregar adheridos. (empresas adheridas)
9) Buscador universal. (buscar cualquier elemento)
10) Eliminador universal. (eliminar cualquier elemento)
11) Editor universal. (editar cualquier elemento)
12) Mostrar empresa. (mostrar todos los datos y elementos de la empresa).

No utilizamos IA para la realización, solamente trabajos prácticos realizados por mi en la materia POO para guiarnos en caso de olvidarnos de cómo implementar algún conocimiento.

Code Blame 256 lines (246 loc) · 16.8 KB Raw

```java
package ejercicio;

import javax.swing.JOptionPane;

public class Ejercicio {
    public static void main(String[] args) {
        String empresa = JOptionPane.showInputDialog("Ingrese el nombre de la empresa: ");
        String direccion_emp = JOptionPane.showInputDialog("Ingrese la direccion de la empresa: ");
        String telefono_emp = JOptionPane.showInputDialog("Ingrese el numero de telefono de la empresa: ");
        Empresa e = new Empresa (empresa, direccion_emp, telefono_emp);
        int contador=0;
        int id_persona = 0;
        while (contador==0){
            int opcion = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el numero de la opcion que desea realizar:"
            + "\n 1) Agregar cliente."
                + "\n 2) Agregar empleado."
                + "\n 3) Agregar reporte."
                + "\n 4) Agregar envio."
                + "\n 5) Agregar paquete."
                + "\n 6) Agregar producto."
                + "\n 7) Agregar vehiculo."
                + "\n 8) Agregar adheridos."
                + "\n 9) Buscador universal."
                + "\n 10) Eliminador universal."
                + "\n 11) Editor universal."
                + "\n 12) Mostrar empresa."));
            switch(opcion){
                case 1:
                    int cant_clientes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de clientes que desea ingresar: "));
                    for (int i = 0; i < cant_clientes; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_cl = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del cliente: "));
                        String habitual = JOptionPane.showInputDialog("¿Es cliente habitual? Ingrese si o no: ");
                        int cant_compras = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de compras del cliente del cliente: "));
                        double presupuesto = Double.parseDouble(JOptionPane.showInputDialog("Ingrese el presupuesto del cliente: "));
                        Cliente c = new Cliente (id_cl, habitual, cant_compras, presupuesto,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarCliente(c);
                    }
                     break;
                     break;
                case 2:
                    int cant_empleados = Integer.parseInt(JOptionPane.showInputDialog("ingrese la cantidad de empleados que desea ingresar"));
                    for (int i = 0; i < cant_empleados; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_em =  Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del empleado: "));
                        String sector = JOptionPane.showInputDialog("ingrese el sector en donde trabaja el empleado");
                        String cargo = JOptionPane.showInputDialog("ingrese el cargo");
                        String rol = JOptionPane.showInputDialog("ingrese el rol del empleado");
                        Empleado empl = new Empleado (id_em,sector, cargo, rol,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarEmpleado(empl);
                    }
                     break;
                case 3:
                    int cant_reportes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de reportes que desea ingresar"));
                    for (int i = 0; i < cant_reportes; i++) {
                        int id_rep = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del reporte: "));
                        String detalle = JOptionPane.showInputDialog("ingrese el detalle del reporte: ");
                        Reporte rep = new Reporte (id_rep, detalle);
                        int cant_ticket=Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de tickets a añadir :"));
                        for(int x=0;<cant_ticket;x++){
                            int idx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del ticket :"));
                            String fechas=JOptionPane.showInputDialog("Ingrese la fecha del pago .");
                            int totalx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el total del pago :"));
                            String pagos=JOptionPane.showInputDialog("Ingrese el metodo de pago :");
                            int idxx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del cliente del ticket :"));
                            int indice=e.buscador_universal(idxx,1);
                            Cliente c=e.clientes.get(indice);
                            Ticket t=new Ticket(idx,fechas,c,totalx,pagos);
                            rep.agregarTicket(t);
                        }
                        e.agregarReporte(rep);
                    }
                     break;
```

(Decidí no agregar todas las capturas del código para no irradiar la carpeta de capturas).

(Capturas sacadas del GitHub del proyecto).

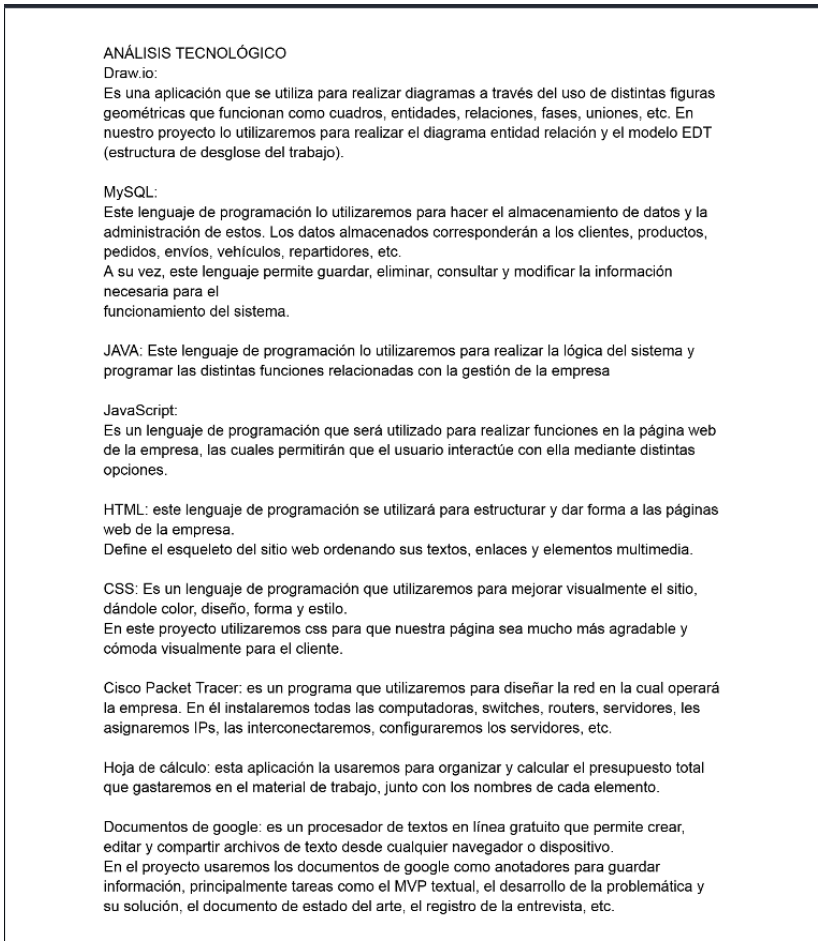
MES DE SEPTIEMBRE:

Durante todo el mes de septiembre, no pude anotar fecha por fecha todas mis tareas realizadas, debido a la aglomeración de estudios y trabajos de otras materias, por lo tanto, la explicación de mis tareas realizadas durante el mes no incluirán su fecha exacta de realización, aunque sí estarán en el orden cronológico correcto.

El día **2/9** tuvimos otro "checkpoint" de lo realizado hasta el momento. Aproximadamente Una hora antes de que tuviéramos que presentar nuestros avances, se me ordenó corregir el Análisis Tecnológico, correspondiente al análisis de sistemas. El mismo había sido realizado previamente por mi compañero Agustin Zowmir, pero tenía errores de redacción, faltantes de información e información errónea. Por lo tanto, cumpli con mi deber a tiempo y quedando así la versión final del análisis tecnológico.

Los principales cambios fueron mejoras en la redacción general, agregar el item ''Javascript'' y "Hojas de cálculo"

Captura sacada del GitHub:



ANÁLISIS TECNOLÓGICO
Draw.io:
Es una aplicación que se utiliza para realizar diagramas a través del uso de distintas figuras geométricas que funcionan como cuadros, entidades, relaciones, fases, uniones, etc. En nuestro proyecto lo utilizaremos para realizar el diagrama entidad relación y el modelo EDT (estructura de desglose del trabajo).

MySQL:
Este lenguaje de programación lo utilizaremos para hacer el almacenamiento de datos y la administración de estos. Los datos almacenados corresponderán a los clientes, productos, pedidos, envíos, vehículos, repartidores, etc.
A su vez, este lenguaje permite guardar, eliminar, consultar y modificar la información necesaria para el
funcionamiento del sistema.

JAVA: Este lenguaje de programación lo utilizaremos para realizar la lógica del sistema y programar las distintas funciones relacionadas con la gestión de la empresa

JavaScript:
Es un lenguaje de programación que será utilizado para realizar funciones en la página web de la empresa, las cuales permitirán que el usuario interactúe con ella mediante distintas opciones.

HTML: este lenguaje de programación se utilizará para estructurar y dar forma a las páginas web de la empresa.
Define el esqueleto del sitio web ordenando sus textos, enlaces y elementos multimedia.

CSS: Es un lenguaje de programación que utilizaremos para mejorar visualmente el sitio, dándole color, diseño, forma y estilo.
En este proyecto utilizaremos css para que nuestra página sea mucho más agradable y cómoda visualmente para el cliente.

Cisco Packet Tracer: es un programa que utilizaremos para diseñar la red en la cual operará la empresa. En él instalaremos todas las computadoras, switches, routers, servidores, les asignaremos IPs, las interconectaremos, configuraremos los servidores, etc.

Hoja de cálculo: esta aplicación la usaremos para organizar y calcular el presupuesto total que gastaremos en el material de trabajo, junto con los nombres de cada elemento.

Documentos de google: es un procesador de textos en línea gratuito que permite crear, editar y compartir archivos de texto desde cualquier navegador o dispositivo.
En el proyecto usaremos los documentos de google como anotadores para guardar información, principalmente tareas como el MVP textual, el desarrollo de la problemática y su solución, el documento de estado del arte, el registro de la entrevista, etc.

Posterior a la devolución del checkpoint, en el cual uno de los principales detalles que nos mencionaron fue la ilegibilidad del Diagrama Entidad Relación (DER), lo primero que realicé fue, justamente, una actualización del mismo. El líder Santiago Goldfarb señaló que debíamos quitar algunas tablas que, en lugar de normalizar, organizar mejor el diagrama y la futura base de datos, hacían más confuso su entendimiento. Por lo tanto, yo me encargue de eso.

Uno de los cambios más importantes, fue la eliminación de la tabla “Detalles_producto”, ya que a raíz de la misma, también habían dos tablas las cuales eran “Colores” y “Materiales”. No solo que, principalmente el color, no son atributos muy importantes para la empresa de logística como tal, sino que también se podían directamente incluir en la tabla misma de “Productos”, como atributos normales, sin necesidad de salir de otras tablas conectadas mediante foreign key.

Por lo tanto, además de eliminar las tablas (“Detalle_productos”, “Colores” y “Materiales”), elimine una tabla “Marcas”, la cual conectaba con la tabla “Productos” mediante foreign key, ya que tampoco es un atributo de importancia tal como para crear una tabla aparte. Directamente pasó a ser un atributo común del producto.

La tabla “Empresa_adherida”, tenía un atributo “rubro” el cual se conectaba con otra tabla de mismo nombre, mediante foreign key. Por el mismo motivo por el cual elimine la tabla “Marca” para los productos, elimine “rubro” también, transformándolo en un atributo directo de la tabla “Empresa_adherida”.

En la tabla “Envios”, existen los atributos “origen” y “destino”, estos se conectaban mediante foreign key con la tabla “Ciudades”. Hasta ahí todo bien, pero la tabla “Ciudades” también conectaba con una tabla “Provincias”, la cual decidimos eliminar, y pasó a ser un atributo directo de la tabla “Ciudades”.

Por otro lado, también conecte la tabla “Envios” con “Cliente” y “Empresa_Adherida”, importante para poder conocer qué empresa lo solicita y a que cliente está destinado.

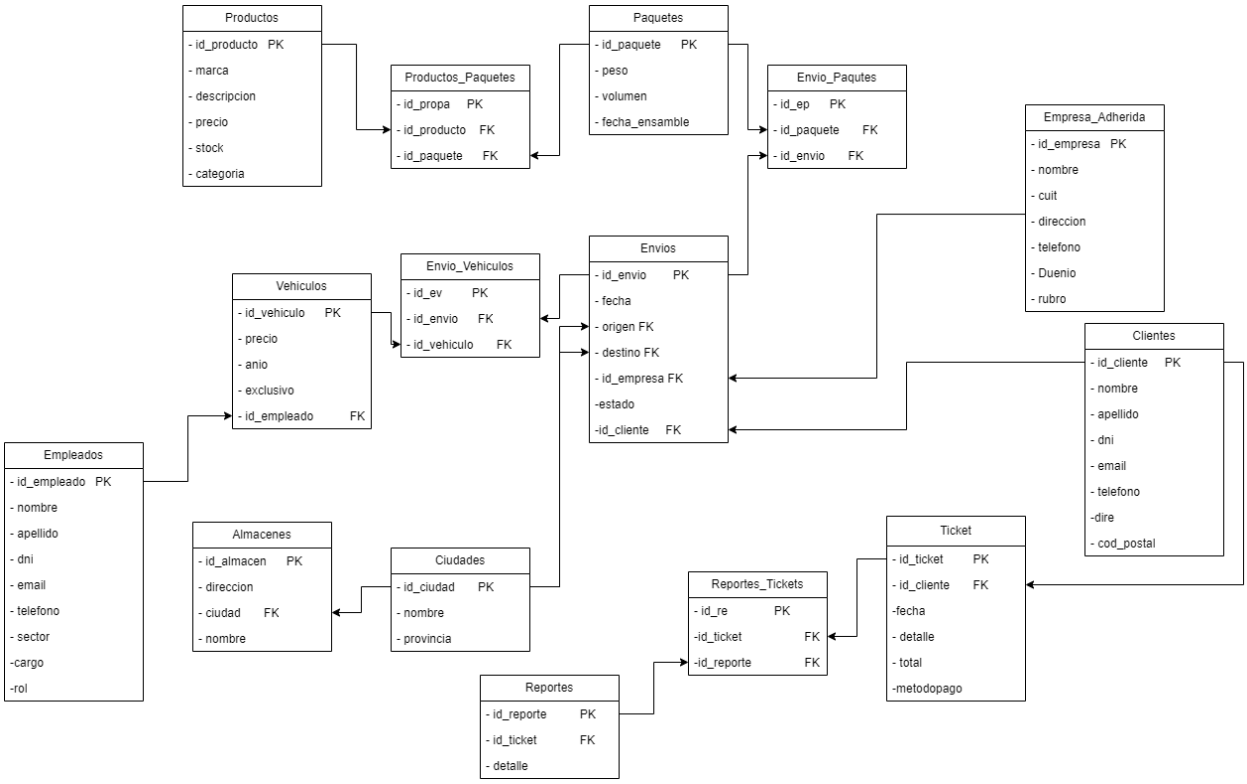
También le agregue el atributo “estado” a la tabla “Envios”, para poder saber si el envío se encuentra en proceso, si llego a destino, si se perdió, etc.

Por el lado de los vehículos, hice algo similar a lo ocurrido con la tabla “Productos”. Existía una tabla llamada “Detalle_vehiculos”, la cual incluía un atributo “exclusivo” el cual tranquilamente se podía incluir directo a la tabla “Vehiculos”.
Además de modificar eso, el atributo “marca” pasó a ser directo de la tabla “Vehículos”, debido a la anterior mencionada eliminación de la tabla con el mismo nombre.

Por último, modifique los nombres de las tablas intermedias, como “Env_pa”, que paso a ser “Envio_paquetes”, “Pro_Pa”, que pasó a ser “Productos_paquetes”, “Env_ve” que paso a ser “Envio_paquetes”, etc.

Por lo que vi en el GitHub, el DER igual tuvo un par de cambios posteriores mínimos, de los cuales no tenía conocimiento. El mio fue eliminado del repositorio y no cuento con mi versión. Aunque, por lo que recuerdo, las diferencias del DER que figura en el repositorio con el que había realizado yo son casi nulas (por ejemplo, yo nunca puse el atributo ‘’dire’’ en la tabla “Clientes”, o la falta del atributo “marca” en vehículos (lo que mencioné, la eliminación de la tabla “marcas” y la transforme en atributo normal).

DER del GitHub, 90%+ modificado por mi:



Un par de días posteriores a esto, realicé las tablas completas de la Base de Datos, en MySQL Workbench 8.0, a partir de la versión final del Diagrama Entidad Relación. Durante la realización de las tablas, mi compañero Benjamin Saragusti me iba señalando si me equivocaba en algo.

Si bien yo tengo muy claro el cómo crear las tablas de una base de datos, lo que ocurrió seguido fue que, al basarme en el diccionario de datos para la realización de la misma, encontrábamos errores en este, los cuales debía corregir yo. Como estoy redactando este texto días después del día mencionado, no recuerdo con certeza qué partes del diccionario de datos tuve que modificar.

Lo que sí recuerdo es que eran errores no muy graves, por ejemplo que un atributo que debía ser de tipo “int” figurase como “decimal”, o que figure “varchar(25)” cuando lo ideal era “varchar(50)”, y más errores de ese estilo.

La base de datos quedo asi:

```
drop database if exists Logistica32;
create database Logistica32;
use Logistica32;

create table Empresa_Adherida(
    ID_empresa int unsigned auto_increment primary key,
    nombre varchar(50),
    cuit varchar(13),
    direccion varchar(100),
    telefono int (10),
    rubro varchar(100),
    duenio varchar(50)
);

create table Empresa_Adherida_eliminada(
    ID_empresa_eliminada int unsigned auto_increment primary key,
    nombre_eliminado varchar(50),
    cuit_eliminado varchar(13),
    direccion_eliminada varchar(100),
    telefono_eliminado int (10),
    rubro_eliminado varchar(100),
    duenio_eliminado varchar(50)
);


create table Productos(
ID_Producto int unsigned auto_increment primary key,
marca varchar(200),
descripcion varchar(100),
precio decimal(10,2),
stock int,
categoria varchar(50)
);

create table Paquetes(
ID_Paquete int unsigned auto_increment primary key,
peso decimal(9,2),
volumen decimal(9,2),
fecha_ensamble date
);
```

```
);

create table Productos_Paquetes(
ID_Propa int unsigned auto_increment primary key,
ID_Producto int unsigned,
ID_Paquete int unsigned,
cantidad int,
foreign key (ID_Producto) references Productos(ID_Producto),
foreign key (ID_Paquete) references Paquetes(ID_Paquete)
);

Create table Ciudades(
ID_Ciudad int unsigned auto_increment primary key,
Nombre varchar(50),
Provincia varchar(50)
);
CREATE TABLE Ciudades_Historico_Cambios (
    ID_Historico INT AUTO_INCREMENT PRIMARY KEY,
    ID_Ciudad INT,
    Nombre_Viejo VARCHAR(50),
    Nombre_Nuevo VARCHAR(50),
    Provincia_Vieja VARCHAR(50),
    Provincia_Nueva VARCHAR(50),
    Fecha_Modificacion DATE
);


create table Clientes(
ID_Cliente int unsigned auto_increment primary key,
Nombre varchar(50),
Apellido varchar(50),
DNI int(8),
email varchar(50),
telefono int (10),
direccion varchar(1000),
cod_postal int(4)
);
```

```
Query 1
Limit to 1000 rows
76  );
77  CREATE TABLE Clientes_Historico_Borrados (
78      ID_Historico INT AUTO_INCREMENT PRIMARY KEY,
79      ID_Cliente INT,
80      Nombre VARCHAR(50),
81      Apellido VARCHAR(50),
82      DNI INT,
83      Fecha_Baja date
84  );
85
86
87
88  create table Envios(
89  ID_Envio int unsigned auto_increment primary key,
90  Fecha date,
91  Origen int unsigned,
92  Destino int unsigned,
93  ID_Empresa int unsigned,
94  estado varchar(50),
95  ID_Cliente int unsigned,
96  foreign key (Origen) references Ciudades(ID_Ciudad),
97  foreign key (Destino) references Ciudades(ID_Ciudad),
98  foreign key (ID_Empresa) references Empresa_Adherida(ID_Empresa),
99  foreign key (ID_Cliente) references Clientes(ID_Cliente)
100 );
101
102 create table Envios_eliminados(
103 ID_Envio_eliminado int unsigned auto_increment primary key,
104 Fecha_eliminado date,
105 Origen_eliminado int unsigned,
106 Destino_eliminado int unsigned,
107 ID_Empresa_eliminado int unsigned,
108 estado_eliminado varchar(50),
109 ID_Cliente_eliminado int unsigned,
110 foreign key (Origen_eliminado) references Ciudades(ID_Ciudad),
111 foreign key (Destino_eliminado) references Ciudades(ID_Ciudad),
112 foreign key (ID_Empresa_eliminado) references Empresa_Adherida(ID_Empresa),
113 foreign key (ID_Cliente_eliminado) references Clientes(ID_Cliente)
114 );
115
```

```
Query 1
Limit to 1000 rows
115
116 create table Empleados(
117 ID_Empleado int unsigned auto_increment primary key,
118 Nombre varchar(50),
119 Apellido varchar(50),
120 Dni int(8),
121 email varchar(100),
122 telefono int (10),
123 sector varchar(100),
124 cargo varchar(100),
125 rol varchar(100)
126 );
127
128 create table Vehiculos (
129 ID_Vehiculo int unsigned auto_increment primary key,
130 Precio int (11),
131 anio int (11),
132 ID_Empleado int unsigned,
133 foreign key (ID_Empleado) references Empleados (ID_Empleado)
134 );
135
136 Create table Envio_Vehiculos(
137 ID_EV int unsigned auto_increment primary key,
138 ID_Envio int unsigned,
139 ID_Vehiculo int unsigned,
140 foreign key (ID_Envio) references Envios(ID_Envio),
141 foreign key (ID_Vehiculo) references Vehiculos(ID_Vehiculo)
142 );
143
144 create table Envio_Paquetes (
145 ID_ep int unsigned auto_increment primary key,
146 ID_Envio int unsigned,
147 ID_Paquete int unsigned,
148 foreign key (ID_Envio) references Envios(ID_Envio),
149 foreign key (ID_Paquete) references Paquetes(ID_Paquete)
150 );
151
```

```
create table Reportes(
ID_Reporte int unsigned auto_increment primary key,
detalle varchar(1000)
);

create table Tickets(
ID_Ticket int unsigned auto_increment primary key,
ID_Cliente int unsigned,
fecha date,
detalle varchar(1000),
total int(11),
metodospago varchar(100),
foreign key (ID_Cliente) references Clientes (ID_Cliente)
);

create table Reportes_Ticket(
ID_rt int unsigned auto_increment primary key,
ID_Reporte int unsigned,
ID_Ticket int unsigned,
foreign key (ID_Reporte) references Reportes (ID_Reporte),
foreign key (ID_Ticket) references Tickets (ID_Ticket)
);

create table Almacenes (
ID_Almacen int unsigned auto_increment primary key,
direccion varchar(50),
ciudad int unsigned,
nombre varchar(50),
foreign key (ciudad) references Ciudades (ID_Ciudad)
);
```

Posteriormente, se le sumaron las columnas “Modelo” y “Marca” a la tabla “vehiculos”, mediante el comando alter table.

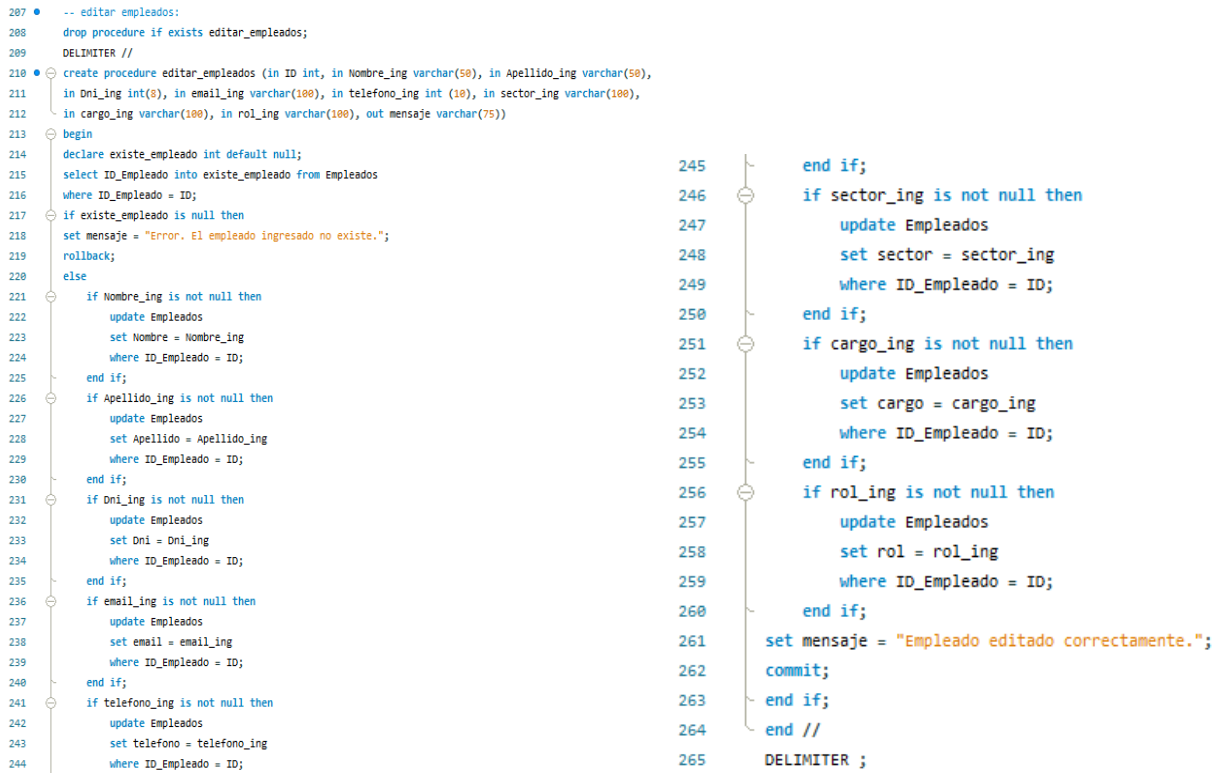
Una vez finalizada la base de datos en su totalidad, se nos asigno realizar tres procedures básicos (insertar, editar y eliminar) para las distintas tablas.
La división fue dos tablas normales y una tabla intermedia por integrante. A mi se me asignaron las tablas “Empleados”, “Vehiculos” y “Envio_vehiculos”.
Por lo tanto durante la segunda semana de septiembre, cumpli con mi mandato y realice los procedures que permiten insertar, editar y eliminar empleados (tres procedures distintos), insertar, editar y eliminar vehiculos (otros tres procedures), insertar, editar y eliminar envio_vehiculos (otros tres procedures), siendo un total de nueve procedures.

Procedure agregar empleados:

```
drop procedure if exists agregar_empleados;
DELIMITER //
create procedure agregar_empleados(in Nombre varchar(50), in Apellido varchar(50), in Dni int(8), in email varchar(100),
in telefono int (10), in sector varchar(100), in cargo varchar(100), in rol varchar(100), out mensaje varchar(75))
begin
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   begin
      SET mensaje = "Error. Operacion cancelada";
```

```
        rollback;
    end;
    start transaction;
    insert into empleados(Nombre, Apellido, Dni, email, telefono, sector, cargo, rol)
    values (Nombre, Apellido, Dni, email, telefono, sector, cargo, rol);
    commit;
    set mensaje = "Empleado registrado correctamente.";
end//
DELIMITER ;
```

Procedure editar empleados (lo pego como imagen porque el código es muy largo):



```
-- editar empleados:
drop procedure if exists editar_empleados;
DELIMITER //
create procedure editar_empleados (in ID int, in Nombre_ing varchar(50), in Apellido_ing varchar(50),
in Dni_ing int(8), in email_ing varchar(100), in telefono_ing int (10), in sector_ing varchar(100),
in cargo_ing varchar(100), in rol_ing varchar(100), out mensaje varchar(75))
begin
declare existe_empleado int default null;
select ID_Empleado into existe_empleado from Empleados
where ID_Empleado = ID;
if existe_empleado is null then
set mensaje = "Error. El empleado ingresado no existe.";
rollback;
else
    if Nombre_ing is not null then
        update Empleados
        set Nombre = Nombre_ing
        where ID_Empleado = ID;
    end if;
    if Apellido_ing is not null then
        update Empleados
        set Apellido = Apellido_ing
        where ID_Empleado = ID;
    end if;
    if Dni_ing is not null then
        update Empleados
        set Dni = Dni_ing
        where ID_Empleado = ID;
    end if;
    if email_ing is not null then
        update Empleados
        set email = email_ing
        where ID_Empleado = ID;
    end if;
    if telefono_ing is not null then
        update Empleados
        set telefono = telefono_ing
        where ID_Empleado = ID;
    end if;
    if sector_ing is not null then
        update Empleados
        set sector = sector_ing
        where ID_Empleado = ID;
    end if;
    if cargo_ing is not null then
        update Empleados
        set cargo = cargo_ing
        where ID_Empleado = ID;
    end if;
    if rol_ing is not null then
        update Empleados
        set rol = rol_ing
        where ID_Empleado = ID;
    end if;
set mensaje = "Empleado editado correctamente.";
commit;
end if;
end //
DELIMITER ;
```

Procedure eliminar empleado:

```
drop procedure if exists eliminar_empleado;
DELIMITER //
create procedure eliminar_empleado (in id_ing int, out mensaje varchar(75))
begin

declare existe_empleado int default null;
declare tiene_vehiculo int default null;
select ID_Empleado into existe_empleado from empleados
where ID_Empleado = id_ing;

select ID_Empleado into tiene_vehiculo from Vehiculos
where ID_Empleado = id_ing;

if tiene_vehiculo is not null then
set mensaje = "Error. El empleado no puede tener un vehiculo si se lo eliminará.";

elseif existe_empleado is not null then
    delete from empleados
    where ID_Empleado = id_ing;
    commit;
    set mensaje = "Empleado eliminado correctamente.";
else
```

```
        set mensaje = "Error. El empleado ingresado no existe.";
        rollback;
end if;
end //
DELIMITER ;
```

(Los demás procedures tenían todos exactamente la misma estructura, solamente modificando los datos correspondientes a cada tabla, por lo que decidí no pegarlos también para no irradiar la carpeta de códigos).

No utilice Inteligencia Artificial para realizar estos procedures, a lo sumo si me olvidaba de cómo hacer algo, lo consultaba en trabajos realizados por mi durante el año en la materia de Base de Datos.

Una vez que todos los integrantes del grupo finalizaron con sus procedures y fueron corroborados por nosotros mismos, el integrante Benjamin Saragusti nos asignó a la semana siguiente (aproximadamente el **17/9**) que cada integrante debía realizar cincuenta inserts por cada tabla (de las que había cada uno realizado los procedures).
De esos cincuenta inserts, cinco debían ser realizados mediante los procedures que hicimos, y los restantes podían ser hardcodeados con el tradicional ‘’insert into values’’.
Por lo tanto, durante esa semana, realice mi total de ciento cincuenta inserts correspondientes a las tablas “Empleados” (50), “Vehiculos” (50) y “Envio_vehiculo” (50).

Los inserts en los cuales se aplican mis procedures quedaron de la siguiente manera:

CALL agregar_empleados('Carlos', 'Gómez', 32451892, 'carlos.gomez01@logistica.com', 11456701, null, null, null, @mensaje);
CALL agregar_empleados('María', 'Fernández', 28912304, 'maria.fernandez02@logistica.com', 11456702, null, null, null, @mensaje);
CALL agregar_empleados('Juan', 'Pérez', 41058291, 'juan.perez03@logistica.com', 11456703, null, null, null, @mensaje);
CALL agregar_empleados('Lucía', 'Martínez', 35781902, 'lucia.martinez04@logistica.com', 11456704, null, null, null, @mensaje);
CALL agregar_empleados('Santiago', 'López', 29384710, 'santiago.lopez05@logistica.com', 11456705, null, null, null, @mensaje);

Los atributos “Sector”, “Cargo” y “Rol” los definí como null debido a que aún no habíamos decidido bien cuales serían dichos atributos.

El día miércoles **23/09**, Benjamin Saragusti nos asigno a mis compañeros Nicolas Rosato y Xiomara Ureña a realizar modificaciones al código de JAVA en el que él había estado trabajando durante la semana. La parte que nos asignó corregir era la correspondiente a las primeras versiones de las ventanas interactivas JFrame correspondientes a los CRUD (sin contar aún con la conexión a la base de datos).

Si bien no cuento con las pruebas originales, recuerdo que las modificaciones consistían principalmente en agregar validaciones básicas en los campos a completar de las tablas, por ejemplo, verificar que los campos no estén vacíos al intentar agregar algún elemento. Además, tuvimos que agregar un contador el cual, al detectar diez veces un error al querer realizar alguna tarea en una misma ventana, cerraba el programa.
Al principio, nuestras validaciones no funcionaron como correspondían, por lo que Goldfarb y Saragusti verificaron el código y encontraron una solución.

Entre los días **26/9** y **30/9** realicé dos tareas prácticamente en simultáneo:
El día **25/7** se me había encargado realizar realizar una documentación con observaciones y un listado explicativo en el que se mostraría cómo funcionaban las ventanas del código de JAVA, pero durante la realización del mismo, descubrí que las ventanas presentaban muchas validaciones y detalles faltantes, así como también los aspectos visuales de las ventanas, los cuales se veían bastante primitivos. Por lo tanto, entre los tres días mencionados anteriormente me dediqué a mejorar los códigos de todas las ventanas de pies a cabeza, en el siguiente orden:

Entre los días **27/9 y 28/9**, comencé utilizando la ventana de gestión de clientes como mi conejillo de indias. Como todas las ventanas tenían más o menos las mismas fallas y detalles a corregir, mi idea fue revisar, corregir, y documentar de pies a cabeza todo el funcionamiento de esta ventana hasta que funcione al 100%, para así después hacer un calco con las demás ventanas.

Al principio, la documentación tenia mas formato de guia o manual para el usuario, pero a medida que iba encontrando los errores, pasó a ser una muestra de todas las cosas que corregí en la ventana clientes (no continué esta documentación con las demás ventanas, debido a que las funcionalidades de las ventanas debían estar terminadas para antes del **30/9**, dia en el que teníamos el punto de control de septiembre, y por lo tanto, realizar a la vez la documentación de todos los cambios me iba a llevar bastante más tiempo).

En la noche del **29/09** me dediqué a realizar todas las validaciones de cada una de las ventanas (Cliente, Adherido, Empleado, Envio, Paquete, Producto, Reporte, Ticket y Vehículo). Además, a varias ventanas les faltaban funcionalidades del CRUD (como Editar, Modificar, Borrar. Las de agregar estaban en todas las ventanas), por lo tanto, agregué todas las funcionalidades básicas de los CRUD que faltaban.
Como lo realice a media noche, lo finalice todo en la madrugada del **30/09**, horas antes del punto de control de septiembre.
Por último, mejore el aspecto de una de las ventanas, más precisamente la de ‘’Gestion de clientes’’, aunque continuaremos con eso luego de la entrega de Septiembre.

Asi se ve el código correspondiente a Agregar cliente, con todas sus validaciones incluidas:

```
    private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {
        if (contador >= 10){
```

```
            JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por
consiguiente el programa se reiniciara");
            System.exit(0);
        }
        String nombre = txtNombre.getText();
        String apellido = txtApellido.getText();
        String dni = txtDni.getText();
        String Email = txtEmail.getText();
        String telefono = txtTelefono.getText();
        String id_cliente=txtCliente.getText();
        String habi=txtHabitual.getText();
        String comp=txtCompras.getText();
        String pesu=txtPresupuesto.getText();

        // CAMPOS VACIOS
        if("".equals(nombre) || "".equals(dni) ||"".equals(apellido) ||"".equals(Email) ||
"".equals(telefono)||"".equals(id_cliente)||"".equals(comp)||"".equals(habi)||"".equals(pesu) ){
            JOptionPane.showMessageDialog(null, "ERROR - Los campos no pueden estar
vacios.");
            contador++;
           return;
        }
        // condicionales Nombre
```

```
        if(!nombre.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")){
            JOptionPane.showMessageDialog(null, "ERROR - El nombre solo puede contener
letras");
            contador++;
            return;
        }
        // condicionales Apellido
        if(!apellido.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")){
            JOptionPane.showMessageDialog(null, "ERROR - El apellido solo puede contener
letras");
            contador++;
            return;
        }
        // condicionales DNI
        if(!dni.matches("[0-9]+") && !dni.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El DNI solo puede contener
numeros.");
            contador++;
            return;
        }
        if(dni.length()!=8){
            JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo 8 digitos");
            contador++;
```

```
            return;
        }
        // condicionales email
        if(!Email.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")){
            JOptionPane.showMessageDialog(null, "ERROR - Ingrese un correo electronico
valido");
            contador++;
            return;
        }
        // condicionales telefono
        if(!telefono.matches("[0-9]+") && !telefono.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono solo puede contener
numeros.");
            contador++;
            return;
        }
        if(telefono.length()!=11 && telefono.length()!=10){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono debe contar con 10 u 11
digitos");
            contador++;
            return;
        }
        // condicionales id_cliente
```

```
        if(!id_cliente.matches("[0-9]+") && !id_cliente.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El ID solo puede contener numeros
enteros.");
            contador++;
            return;
        }
        int num2=Integer.parseInt(id_cliente);
        if(gestion.buscador_universal(num2, 1) != -1){
            JOptionPane.showMessageDialog(null, "ERROR - El ID ingresado ya existe.");
            contador++;
            return;
        }
        // condicionales habitual
        habi = habi.toLowerCase();
        if (!habi.matches("si|no") && !habi.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - En el campo 'Habitual', solo se puede
ingresar 'Si' o 'No'");
            contador++;
            return;
        }
        // condicionales cantidad de compras
        if(!comp.matches("[0-9]+") && !comp.equals("")){
```

```
            JOptionPane.showMessageDialog(null, "ERROR - La cantidad de compras solo puede
contener numeros enteros.");
            contador++;
            return;
        }
        // condicionales presupuesto
        if (!pesu.matches("[0-9]+([.][0-9]+)?") && !pesu.equals("")) {
            JOptionPane.showMessageDialog(null, "ERROR - El presupuesto solo puede contener
un numero decimal (usar . y no ,).");
            contador++;
            return;
        }
        int num=Integer.parseInt(comp);
        double num3;
        num3 = Double.parseDouble(pesu);

        Cliente nueva_persona = new Cliente(num2,habi, num,  num3,
nombre,apellido,dni,Email,telefono);
        gestion.agregarCliente(nueva_persona);
        JOptionPane.showMessageDialog(null, "Cliente registrado/a exitosamente.");
        txtNombre.setText("");
        txtApellido.setText("");
        txtDni.setText("");
```

```
        txtEmail.setText("");
        txtTelefono.setText("");
        txtCliente.setText("");
        txtHabitual.setText("");
        txtCompras.setText("");
        txtPresupuesto.setText("");
    }
```

El enlace a continuación, corresponde a la documentación de funcionamiento que había empezado el día anterior. Si bien, como mencione, no contiene todo el contenido que le corresponde, contiene captura de todos los posibles mensajes de error que pueden aparecer, los cuales funcionan gracias a las validaciones que realice (se usa el ejemplo de la ventana “Clientes”). Además, muestra capturas comparativas del aspecto de esa misma ventana, antes y después de mis modificaciones.

Pruebas Codigo POO - Proyecto Logistica - Santiago Agustín Pedraza Clérici

Ese mismo día, tras el punto de control, nos dijeron que todo estaba relativamente bien, pero con algunos detalles a tener en cuenta. Primero que nada, teníamos que empezar de una vez con el sitio web HTML, por otro lado, el profesor Gabriel Tuinyla nos dijo que teníamos muy pocos procedures. No bastaba solo con Agregar, Borrar y Editar, teníamos que hacer mínimo 10-15 procedures por tabla. Por último, nos dijeron que empezáramos con la conexión de Java a MySQL.

Luego de que se me encargase realizar los procedures restantes para la cantidad mínima solicitada, entre los días **3/10** y **6/10** realice todos los restantes correspondientes a las tablas Empleados y Vehículos, las mismas a las cuales realice los primeros procedures previos a la entrega de septiembre.

Para la tabla empleados, realice los siguientes trece procedures:

```
202     -- mostrar empleados
203 •   drop procedure if exists mostrar_empleados;
204     DELIMITER //
205 •   create procedure mostrar_empleados()
206     begin
207     select * from empleados;
208     end //
209     DELIMITER ;
210 •   call mostrar_empleados();
211
212     -- buscar empleados
213     delimiter //
214 •   create procedure buscar_empleado(in p_id_empleado int)
215         begin
216             select * from empleados where ID_Empleado=p_id_empleado;
217         end//
218     delimiter;
219
220     -- mostrar empleado mas viejo
221     drop procedure if exists empleado_mas_viejo;
222     DELIMITER //
223 •   create procedure empleado_mas_viejo()
224     begin
225     select * from Empleados
226     where dni = (select min(dni) from empleados);
227     end //
228     DELIMITER ;
```

```sql
-- mostrar empleados del sector ‘’transporte’’
drop procedure if exists empleados_transporte;
DELIMITER //
create procedure empleados_transporte()
begin
select * from Empleados
where Sector = "Transporte";
end //
DELIMITER ;

-- mostrar empleados del sector “deposito”
drop procedure if exists empleados_deposito;
DELIMITER //
create procedure empleados_deposito()
begin
select * from Empleados
where Sector = "Deposito";
end //
DELIMITER ;

-- mostrar empleados del sector “administracion”
drop procedure if exists empleados_administracion;
DELIMITER //
create procedure empleados_administracion()
begin
select * from Empleados
where Sector = "Administracion";
end //
DELIMITER ;
```

```sql
-- mostrar empleados del sector “atencion al cliente”
drop procedure if exists empleados_atencion;
DELIMITER //
create procedure empleados_atencion()
begin
select * from Empleados
where Sector = "Atencion al cliente";
end //
DELIMITER ;

-- mostrar empleados del sector “finanzas”
drop procedure if exists empleados_finanzas;
DELIMITER //
create procedure empleados_finanzas()
begin
select * from Empleados
where Sector = "Finanzas";
end //
DELIMITER ;

-- mostrar empleados del sector “mantenimiento”
drop procedure if exists empleados_mantenimiento;
DELIMITER //
create procedure empleados_mantenimiento()
begin
select * from Empleados
where Sector = "Mantenimiento";
end //
DELIMITER ;
```

```sql
-- mostrar empleados con rol “lider”
drop procedure if exists empleados_lideres;
DELIMITER //
create procedure empleados_lideres()
begin
select * from Empleados
where Rol = "Lider";
end //
DELIMITER ;

-- mostrar empleados con rol “empleado comun”
drop procedure if exists empleados_comunes;
DELIMITER //
create procedure empleados_comunes()
begin
select * from Empleados
where Rol = "Empleado comun";
end //
DELIMITER ;

-- mostrar empleados con rol “empleado con beneficios”
drop procedure if exists empleados_beneficiados;
DELIMITER //
create procedure empleados_beneficiados()
begin
select * from Empleados
where Rol= "Empleado con beneficios";
end //
DELIMITER ;
```

```sql
-- mostrar empleados con rol “empleado temporal”
drop procedure if exists empleados_temporales;
DELIMITER //
create procedure empleados_temporales()
begin
select * from Empleados
where Rol= "Empleado temporal";
end //
DELIMITER ;
```

Sumados estos 13 procedures a los 3 realizados previamente correspondientes a “Agregar”, “Editar” y “Eliminar”, queda un total de 16 procedures para la tabla Empleados.

Lo mismo hice con la tabla vehículos, solo que en lugar de 13 procedures nuevos, realice solo 8. Sumándole los 3 previos, nos da un total de 11 procedures para la tabla.

Nuevos procedures de la tabla Vehículos (además de buscar y mostrar que son iguales a los de Empleados):

```
-- buscar vehículos mediante el id del empleado
drop procedure if exists buscar_vehiculo_por_empleado;
DELIMITER //
create procedure buscar_vehiculo_por_empleado(in id_emp_ing int)
begin
select * from vehículos
where ID_Empleado = id_emp_ing;
end //
DELIMITER ;

-- buscar vehículos mediante la marca
drop procedure if exists buscar_vehiculo_por_marca;
DELIMITER //
create procedure buscar_vehiculo_por_marca(in marca_ing varchar(100))
begin
select * from vehículos
where Marca = marca_ing;
end //
DELIMITER ;

-- buscar vehículos mediante el modelo
drop procedure if exists buscar_vehiculo_por_modelo;
DELIMITER //
create procedure buscar_vehiculo_por_modelo(in modelo_ing varchar(100))
begin
select * from vehículos
where Modelo = modelo_ing;
end //
DELIMITER ;
```

```
362 •   -- mostrar vehiculos modernos (fabricados en 2020 o posterior)
363     drop procedure if exists vehiculos_modernos;
364     DELIMITER //
365 •   create procedure vehiculos_modernos()
366     begin
367     select * from Vehiculos
368     where anio >= 2020;
369     end //
370     DELIMITER ;
371
372 •   -- mostrar vehiculos economicos (precio menor o igual a 15.000.000)
373     drop procedure if exists vehiculos_economicos;
374     DELIMITER //
375 •   create procedure vehiculos_economicos()
376     begin
377     select * from Vehiculos
378     where precio <= 15000000;
379     end //
380     DELIMITER ;
```

El día 7/10, Goldfarb y Saragusti nos asignaron a cada integrante realizar de 1 a 2 clases DAO y las ventanas correspondientes con nuevas condiciones, para el programa en JAVA. A mi me asignaron las clases DAO y las ventanas de Reporte y Ticket.

Las condiciones que me dio Saragusti para las ventanas eran las siguientes:

Ventana reporte:
Agregar, buscar(por id),editar, borrar, mostrar
Ademas:
Agregar tickets
Mostrar tickets
Borrar 1 ticket
Borrar todos los tickets
Preguntenme para mas profundidad en estos
Ventana tickets
Buscar,mostrar,agregar, borrar,editar
Ademas de borrar todos los tickets usando id cliente

Comencé con la actividad el día **10/10**, finalizando las clases DAO ese día, pero se me dificultaba mucho entender las condiciones de las ventanas. Por un lado, consulté al líder

Goldfarb si tenía que incluir los demás procedimientos extras (además de los del CRUD) en las ventanas, pero me dijo que no. Por otro lado, le pedí a Saragusti (quien puso las condiciones de las ventanas) que me lo explique bien. Me mandó un audio explicando todo y por lo tanto, el dia **11/10** pude realizar las ventanas correctamente .

Clase ReporteDAO :



```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
/**
 *
 * @author Redes-20
 */
public class ReporteDAO {
    public void insertar(Reporte r) throws SQLException {
    String consulta_sql = "CALL agregar_reportes(?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setString(1, r.getDetalle());

        sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

        String mensajeRespuesta = sentencia_sql.getString(2);
        System.out.println("Respuesta del servidor: " + mensajeRespuesta);
        System.out.println("Reporte guardado exitosamente");

    } catch (SQLException ex) {
        System.out.println("ERROR :" + ex.getMessage());
    }
}
    public  ArrayList<Reporte> listar(){
        ArrayList<Reporte> reportes=new ArrayList<>();
        String consulta_sql="call mostrar_reportes()";

            Connection conexion=Conexion.conectar();
            try {
                CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
                ResultSet resultado=sentencia_sql.executeQuery();
                while(resultado.next()){
                    Reporte rep=new Reporte();
```

```
                    rep.setIdReporte(resultado.getInt("ID_Reporte"));
                    rep.setDetalle(resultado.getString("detalle"));

                    reportes.add(rep);
                    System.out.println("------------------");
                }
            } catch (SQLException ex) {
                System.out.println("ERROR "+ex.getMessage());
            }
            return reportes;


        }
    public Reporte buscar(int id) {
     String consulta_sql = "call buscar_reportess(?)";
     Connection conexion = Conexion.conectar();

    try {
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
        sentencia_sql.setInt(1, id);
        ResultSet resultado = sentencia_sql.executeQuery();

        if (resultado.next()) {
            Reporte rep=new Reporte();
                    rep.setIdReporte(resultado.getInt("ID_Reporte"));
                    rep.setDetalle(resultado.getString("detalle"));

            System.out.println("------------------");
            return rep;
        } else {
            System.out.println("No se encontró ningún reporte con el ID: " + id);
        }
    } catch (SQLException ex) {
        System.out.println("ERROR " + ex.getMessage());
    }
    return null;
}
    public void eliminar(int id) {
    String consulta_sql = "call eliminarReporte(?, ?)";

    try {
        Connection conexion = Conexion.conectar();
```

```
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, id);

            sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

            sentencia_sql.executeUpdate();

            String respuesta = sentencia_sql.getString(2);
            System.out.println("Respuesta de la BD: " + respuesta);
            System.out.println("Registro eliminado");

        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
    }

    public void actualizar(Reporte rep) {
     String consulta_sql = "call editar_reporte(?, ?, ?)";

     try {
         Connection conexion = Conexion.conectar();
         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

         sentencia_sql.setInt(1, rep.getIdReporte());
         sentencia_sql.setString(2, rep.getDetalle());

         sentencia_sql.registerOutParameter(3, java.sql.Types.VARCHAR);

         sentencia_sql.executeUpdate();

         String respuesta = sentencia_sql.getString(3);
         System.out.println("Respuesta de la BD: " + respuesta);
         System.out.println("Datos actualizados correctamente");

     } catch (SQLException ex) {
         System.out.println("Error " + ex.getMessage());
     }
    }

     public void mostrar_tickets(int id){
         String consulta_sql="call Tickets_reporte(?)";

             Connection conexion=Conexion.conectar();
```

```
             try {
                 CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
                 sentencia_sql.setInt(1, id);
                 ResultSet resultado=sentencia_sql.executeQuery();
                 while(resultado.next()){
                     int id_ticket=resultado.getInt("ID_Ticket");
                     int id_cliente=resultado.getInt("ID_Cliente");
                     String fecha=resultado.getString("fecha");
                     String detalle=resultado.getString("detalle");
                     int total=resultado.getInt("total");
                     String pago=resultado.getString("metodospago");
                     JOptionPane.showMessageDialog(null, "ID del ticket: "+id_ticket+"\n ID del cliente: "+id_cliente+"\n Fecha: " + fecha+"\n Detalle: " + detalle+"\n Total: "+total+"\n Metodo de pago: " +
                     System.out.println("------------------");
                 }
             } catch (SQLException ex) {
                 System.out.println("ERROR "+ex.getMessage());
             }
     }
     public void agregar_ticket(int id_reporte, int id_ticket){
         String consulta_sql = "CALL agregarReportes_Ticket(?, ?, ?)";

     try {
         Connection conexion = Conexion.conectar();
         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

         sentencia_sql.setInt(1, id_reporte);
         sentencia_sql.setInt(2, id_ticket);

         sentencia_sql.registerOutParameter(3, java.sql.Types.VARCHAR);

         sentencia_sql.executeUpdate();

         String mensajeRespuesta = sentencia_sql.getString(8);
         System.out.println("Respuesta del servidor: " + mensajeRespuesta);

     } catch (SQLException ex) {
         System.out.println("ERROR :" + ex.getMessage());
     }
     }

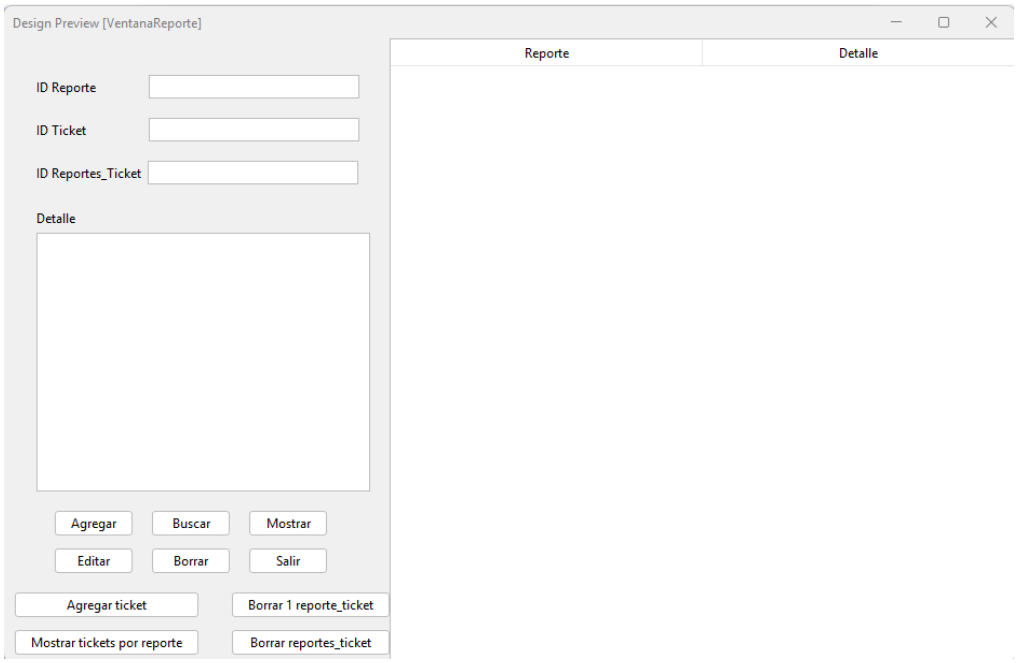
     public void eliminar_reporteticket(int id_rep) {
     String consulta_sql = "call eliminar_ReportesTicket(?, ?)";

     try {
```

```
174     try {
175         Connection conexion = Conexion.conectar();
176         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
177
178         sentencia_sql.setInt(1, id_rep);
179
180         sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);
181
182         sentencia_sql.executeUpdate();
183
184         String respuesta = sentencia_sql.getString(2);
185         System.out.println("Respuesta de la BD: " + respuesta);
186
187     } catch (SQLException ex) {
188         System.out.println("ERROR: " + ex.getMessage());
189     }
190 }
191
192
193     public void eliminar_todosreportesticket(int id_rep) {
194     String consulta_sql = "call eliminar_ReportesTicket2(?)";
195
196     try {
197         Connection conexion = Conexion.conectar();
198         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
199
200         sentencia_sql.setInt(1, id_rep);
201         sentencia_sql.executeUpdate();
202
203
204     } catch (SQLException ex) {
205         System.out.println("ERROR: " + ex.getMessage());
206     }
207 }
208 }
```

La clase TicketDAO tiene el mismo formato, por lo que no incluiré capturas de ese código para no irradiar la carpeta con más capturas.

Ventana Reporte (Preview):



Código para agregar ticket:

```
410     private void btnAddTicketActionPerformed(java.awt.event.ActionEvent evt) {
411         if (contador >= 10){
412             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
413             System.exit(0);
414         }
415         String id_ticket = txtTicket.getText();
416         String id_reporte = txtReporte.getText();
417          if("".equals(id_ticket)|| "".equals(id_reporte) ){
418              JOptionPane.showMessageDialog(null, "ERROR - Los campos de 'ID Reporte' e 'ID Ticket' no pueden estar vacios.");
419              contador++;
420             return;
421         }
422         if(!id_ticket.matches("[0-9]+")){
423             JOptionPane.showMessageDialog(null, "ERROR - El ID del ticket solo puede contener numeros.");
424             contador++;
425             return;
426         }
427         if(!id_reporte.matches("[0-9]+")){
428             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
429             contador++;
430             return;
431         }
432         int idTicket=Integer.parseInt(id_ticket);
433         int idReporte=Integer.parseInt(id_reporte);
434 
435         dao_reporte.agregar_ticket(idReporte, idTicket);
436     }
```

Código para mostrar ticket mediante ID_Reporte:

```
387     private void btnMostrarTicketsActionPerformed(java.awt.event.ActionEvent evt) {
388         if (contador >= 10){
389             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
390             System.exit(0);
391         }
392         String id = txtReporte.getText();
393 
394          if("".equals(id)){
395              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
396              contador++;
397             return;
398         }
399         if(!id.matches("[0-9]+")){
400             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
401             contador++;
402             return;
403         }
404 
405         int id_reporte=Integer.parseInt(id);
406 
407         dao_reporte.mostrar_tickets(id_reporte);
408     }
```

Código para borrar un reporte_ticket:

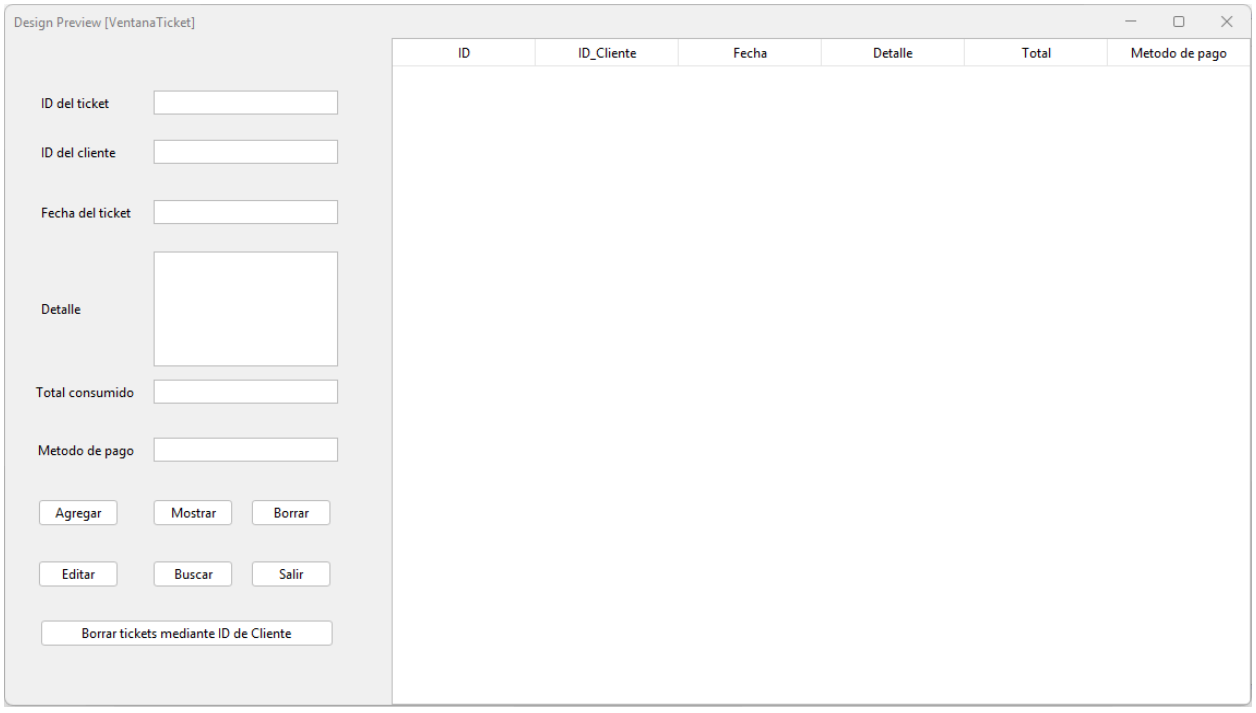
```
460     private void btnBorrar1ReporteActionPerformed(java.awt.event.ActionEvent evt) {
461         if (contador >= 10){
462             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
463             System.exit(0);
464         }
465         String id = txtReporte.getText();
466          if("".equals(id)){
467              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
468              contador++;
469             return;
470         }
471         if(!id.matches("[0-9]+")){
472             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
473             contador++;
474             return;
475         }
476 
477         int id_reporte=Integer.parseInt(id);
478 
479         dao_reporte.eliminar_reporteticket(id_reporte);
480     }
```

Código para borrar todos los reportes_tickets de 1 ID_Reporte:

```
438     private void btnBorrarReportesActionPerformed(java.awt.event.ActionEvent evt) {
439         if (contador >= 10){
440             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
441             System.exit(0);
442         }
443         String id = txtReporte.getText();
444          if("".equals(id)){
445              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
446              contador++;
447             return;
448         }
449         if(!id.matches("[0-9]+")){
450             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
451             contador++;
452             return;
453         }
454 
455         int id_reporte=Integer.parseInt(id);
456 
457         dao_reporte.eliminar_todosreportesticket(id_reporte);
458     }
```

Los códigos CRUD son como los que hice para las validaciones el **30/9.**

Ventana Ticket (Preview):



Código para borrar Tickets mediante ID_Cliente:

```
420     private void btnBorrarPorClienteActionPerformed(java.awt.event.ActionEvent evt) {
421         if (contador >= 10){
422             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
423             System.exit(0);
424         }
425
426         String id = txtCliente.getText();
427         if("".equals(id)){
428             JOptionPane.showMessageDialog(null, "ERROR - No pueden estar vacios los campos");
429             contador++;
430             return;
431         }
432         if(!id.matches("[0-9]+")){
433             JOptionPane.showMessageDialog(null, "ERROR - El ID del cliente solo puede contener numeros.");
434             contador++;
435             return;
436         }
437
438         int id_cliente=Integer.parseInt(id);
439
440         Cliente c = dao_cliente.buscar(id_cliente);
441
442         if (c == null) {
443             JOptionPane.showMessageDialog(null,
444                 "No existe un cliente con ese ID.");
445             return;
446         }
447
448         dao.eliminar_porCliente(id_cliente);
449         actualizarTabla();
450     }
```

Los códigos CRUD son como los que hice para las validaciones el **30/9.**

ACLARACIÓN DE SALTO DE FECHA: aquí se ve un salto importante de fechas (**10/10 - 19/10**). Esto se debió a que me enfoque todos esos días al proyecto que mandó aparte el profesor Gabriel Tuinyla para la materia Proyecto Informático 2.

Durante esa semana, Saragusti nos asigno hacer cada uno partes de la pagina HTML, del codigo JAVA (ya conectado a HTTP, no me ofrecieron participar en eso y, de un dia para el otro, ya estaba todo hecho sin que se me informe), y pisos de la maqueta virtual.

A mi se me asigno realizar esto:

Pedraza:
10-REGISTRAR TRANSPORTISTA (Admin)
11-CONSULTAR TRANSPORTISTA (Admin)
-CONSULTAR TRANSPORTISTA (Cliente) (incluye JAVA y HTML)

Pedraza:
9,10,11 (pisos de la maqueta)

Departamento	Dimensiones (ancho x largo, alto 2.8m)	PCs	Teléfonos IP	Impresoras red	Plotters	Impresoras 3D	Cámaras IP
9. Sistemas y Trazabilidad (RFID/GPS	17.6m x 11.7m (alto 2.8m)	28	28	1	0	2	3
10. Seguridad Patrimonial	5.0m x 5.0m (alto 2.8m)	23	23	1	2	1	3
11. Recursos Humanos	16.1m x 27.0m (alto 2.8m)	115	115	1	0	0	3

El día **20/10,** durante un ratito en la hora de laboratorio, realice el código del servidor web en Java, correspondiente a la función de agregar empleados. Utilice una plantilla que ya había realizada previamente en el mismo código, para agregar vehículos. Solo modifique las partes necesarias para adaptarlo a Empleados.

```
servidor.createContext("/api/Empleados/crear", new HttpHandler() {
  @Override
  public void handle(HttpExchange intercambio) throws IOException {
    if (!intercambio.getRequestMethod().equals("POST")) {
      enviarRespuesta(intercambio, 405,
          "{\"error\":\"Metodo no permitido\"}");
      return;
    }
    // Leer los datos que llegan desde el formulario.
    java.io.InputStream entrada = intercambio.getRequestBody();
    java.io.ByteArrayOutputStream buffer =
        new java.io.ByteArrayOutputStream();
    int dato;
    while ((dato = entrada.read()) != -1) {
      buffer.write(dato);
    }
```

```
String cuerpo = new String(
        buffer.toByteArray(),
        StandardCharsets.UTF_8
);
// Separar los datos recibidos.
java.util.Map<String, String> datos =
        new java.util.HashMap<String, String>();
String[] parametros = cuerpo.split("&");
for (String parametro : parametros) {
    String[] partes = parametro.split("=", 2);
    String clave = java.net.URLDecoder.decode(
            partes[0], "UTF-8"
    );
    String valor = "";
    if (partes.length > 1) {
        valor = java.net.URLDecoder.decode(
                partes[1], "UTF-8"
        );
    }
    datos.put(clave, valor);
}
// Obtener los valores del formulario.
String Nombre = datos.get("Nombre");
```

```
String Apellido = datos.get("Apellido");
String Dni = datos.get("Dni");
String Email = datos.get("email");
String Telefono = datos.get("telefono");
String Sector = datos.get("sector");
String Cargo = datos.get("cargo");
String Rol = datos.get("rol");
// Validar los campos.
if (Nombre == null
        || Apellido == null
        || Dni == null
        || Email == null
        || Telefono == null
        || Sector == null
        || Cargo == null
        || Rol == null
        || Nombre.trim().isEmpty()
        || Apellido.trim().isEmpty()
        || Dni.trim().isEmpty()
        || Email.trim().isEmpty()
        || Telefono.trim().isEmpty()
        || Sector.trim().isEmpty()
        || Cargo.trim().isEmpty()
```

```
            || Rol.trim().isEmpty()
            ) {
        enviarRespuesta(intercambio, 400,
                "{\"error\":\"Todos los campos son obligatorios\"}");
        return;
    }
    int dni;
    int telefono;
                try {
        dni = Integer.parseInt(Dni);
        telefono = Integer.parseInt(Telefono);
    } catch (NumberFormatException e) {
        enviarRespuesta(intercambio, 400,
                "{\"error\":\"El DNI y telefono deben ser numeros enteros\"}");
        return;
    }
    // Preparar la consulta SQL.
   String sql = "CALL agregar_empleado(?, ?, ?, ?, ?, ?, ?, ?, ?)"; // Mantiene los 7 '?'
try (
    Connection conexion = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/logistica32",
            "root",
            ""
```

```
    );
     CallableStatement sentencia = conexion.prepareCall(sql);
) {
    sentencia.setString(1, Nombre);
    sentencia.setString(2, Apellido);
    sentencia.setInt(3, dni);
    sentencia.setString(4, Email);
    sentencia.setInt(5, telefono);
    sentencia.setString(6, Sector);
    sentencia.setString(7, Cargo);
    sentencia.setString(8, Rol);

    // 1. Corregido: Registrar el séptimo parámetro como SALIDA (Ejemplo si devuelve Texto)
    sentencia.registerOutParameter(7, java.sql.Types.VARCHAR);

    sentencia.execute();

    // 2. Opcional: Puedes leer lo que te devolvió la base de datos
    String respuestaBD = sentencia.getString(7);

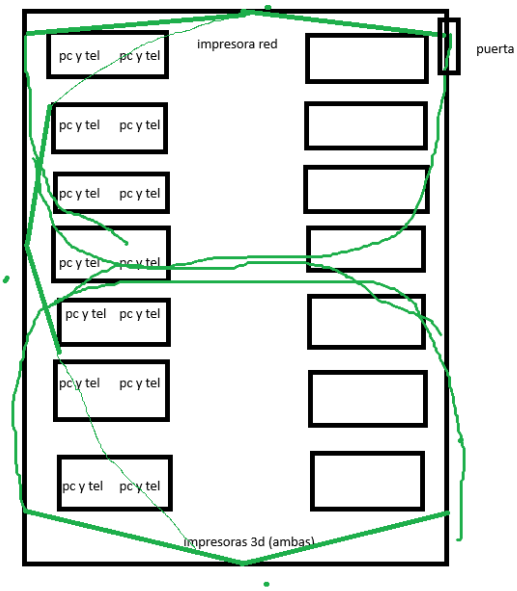
    enviarRespuesta(intercambio, 201,
            "{\"mensaje\":\"Empleado guardado correctamente\", \"detalle\":\"" + respuestaBD +
"\"}");
```

```
} catch (SQLException e) {
    e.printStackTrace();
    enviarRespuesta(intercambio, 500,
            "{\"error\":\"Error al guardar en la base de datos\"}");
}


            }
        });
```

El día **22/10** en mi casa, tras haber terminado finalmente con el proyecto aparte, decidí comenzar con lo que era el diseño de la maqueta virtual. Si bien era consciente de que no era lo más urgente, ya tenía la cabeza quemada de HTML y CSS, por lo que decidí realizar esto otro primero para despejar la mente.
Saragusti me aclaró algunas cosas antes de realizar los diseños: se realizarían en la página Sketchup y que los pisos pares debían llevar router.

Primero diseñe un boceto muy primitivo en Paint, pero que me sirvió bastante (seria la habitacion vista de arriba y sin techo):



Yo no tenía absolutamente ni idea de cómo usar Sketchup, era la primera vez que escuchaba hablar de él. Por lo tanto, para empezar con el diseño del piso 9 pedí un montón de ayudas a la IA (ChatGPT), ayudas como por ejemplo “cómo mover la cámara”, “cómo hacer paredes y techo”, “cómo buscar componentes”, “cómo hacer puertas” y “cómo achicar y ocultar objetos”. Con esas ayudas y el boceto, pude realizar todo el piso 9 ese mismo día, quedando así:



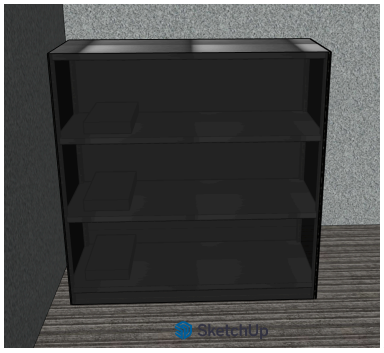
Ahi se ve una impresora 3D y PCs.



Una impresora 3D, una impresora comun de red, PCs y la puerta.



Las capturas simulan ser sacadas desde las 3 cámaras que se encuentran dentro de la habitación.



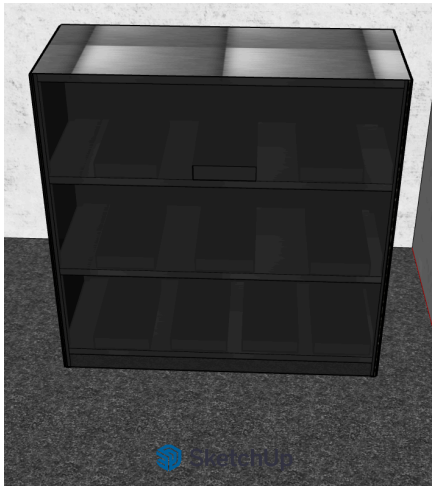
Acá van los switches (3).

El día **23/10** realicé los pisos 11 y 10 respectivamente.

Para realizar el piso 11, no tuve dificultades ni necesite de bocetos ni de ayudas de la IA o vídeos.

Piso 11:





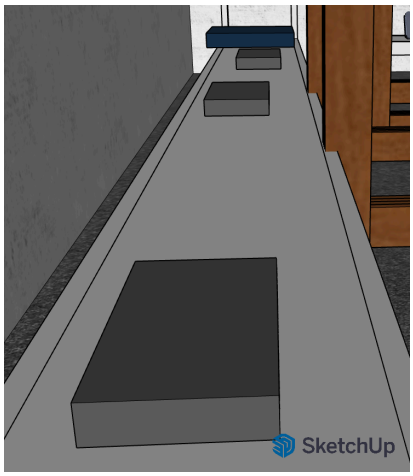
Switches (10)

Tuve una dificultad cuando quise realizar la habitación 10, ya que no sabía cómo meter 23 PCs en una habitación 5x5 mt. Pero después investigue sobre como se veían las habitaciones de seguridad patrimonial, y vi imágenes donde los monitores de las PCs están una encima de otra, en filas y columnas, por lo tanto lo realice de esa manera, usando estanterías para poner las PCs.

Piso 10:







Switches (3) y Router (1)

El día **26/10,** mientras miraba como iba quedando el proyecto, me di cuenta que las ventanas del programa en JAVA estaban bastante desprolijas, con los labels, textfields y botones muy desordenados, y con tamaños distintos. Por lo tanto, le consulté al líder Goldfarb y a mi compañero Saragusti si podía realizar modificaciones con el fin de hacer las ventanas más

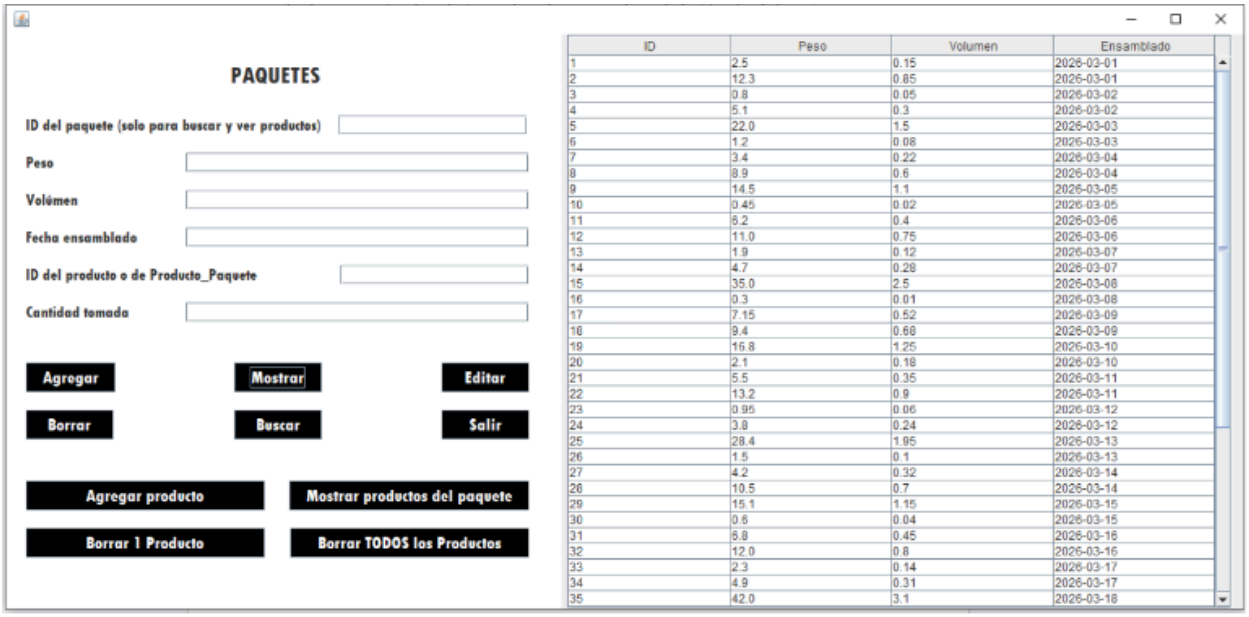
cómodas visualmente y, por lo tanto, más fáciles de utilizar. Me dijeron que sí, y por ende, ese mismo día comencé con eso.

Esto transcurrió durante el horario de Base De Datos. Más tarde, cuando llegué a mi casa quise avanzar, pero tuve un problema: mi versión de NetBeans no tenía incluidas las fuentes de letras utilizadas en las ventanas originales, por lo que se desemparejaron y volvieron a verse desprolijas, asi que decidi trabajar en eso solo con las PCs del colegio.

Al día siguiente, el **27/10,** terminé de mejorar las ventanas que me faltaban.

Todas tienen la misma dinámica. Lo que si, Saragusti me dijo que no modificara la ventana inicial (en la que inicias sesión). Supongo que la modificara el.

Ejemplo de ventana corregida por mi:



ID	Peso	Volumen	Ensamblado
1	2.5	0.15	2026-03-01
2	12.3	0.85	2026-03-01
3	0.8	0.05	2026-03-02
4	5.1	0.3	2026-03-02
5	22.0	1.5	2026-03-03
6	1.2	0.08	2026-03-03
7	3.4	0.22	2026-03-04
8	8.9	0.6	2026-03-04
9	14.5	1.1	2026-03-05
10	0.45	0.02	2026-03-05
11	6.2	0.4	2026-03-06
12	11.0	0.75	2026-03-06
13	1.9	0.12	2026-03-07
14	4.7	0.28	2026-03-07
15	35.0	2.5	2026-03-08
16	0.3	0.01	2026-03-08
17	7.15	0.52	2026-03-09
18	9.4	0.68	2026-03-09
19	16.8	1.25	2026-03-10
20	2.1	0.18	2026-03-10
21	5.5	0.35	2026-03-11
22	13.2	0.9	2026-03-11
23	0.95	0.06	2026-03-12
24	3.8	0.24	2026-03-12
25	28.4	1.95	2026-03-13
26	1.5	0.1	2026-03-13
27	4.2	0.32	2026-03-14
28	10.5	0.7	2026-03-14
29	15.1	1.15	2026-03-15
30	0.6	0.04	2026-03-15
31	6.8	0.45	2026-03-16
32	12.0	0.8	2026-03-16
33	2.3	0.14	2026-03-17
34	4.9	0.31	2026-03-17
35	42.0	3.1	2026-03-18

Posteriormente, una vez que llegué a mi casa, realice los triggers que tenía pendientes desde hacía algunas semanas. No eran urgentes, pero tenía que hacerlos igual.

Realice los triggers (11 en total) correspondientes a las mismas tablas de las cuales había realizado anteriormente los procedures (Empleados, que realicé 6 y Vehículos, que realicé 5).

Si bien las estructuras de los triggers como tal las codifique yo mismo, con ayuda de trabajos realizados por mi anteriormente en la materia de base de datos, las ideas sobre qué funcionamientos hacer, se los pedí a la Inteligencia Artificial (Gemini), ya que no estaba tan lúcido mentalmente en ese momento como para andar pensando en ellos.

Triggers Empleados:

```
-- trigger empleados: verificar que el cargo corresponda al sector transporte
drop trigger if exists verificar_sector_transporte;
DELIMITER //
create trigger verificar_sector_transporte
before insert on empleados
for each row
begin
if new.Cargo != "Transportista" and new.Cargo != "Repartidor" and new.Cargo != "Supervisor de transporte" and new.Sector = "Transporte" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector deposito
drop trigger if exists verificar_sector_deposito;
DELIMITER //
create trigger verificar_sector_deposito
before insert on empleados
for each row
begin
if new.Cargo != "Operario" and new.Cargo != "Repositor" and new.Sector = "Deposito" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

```
-- trigger empleados: verificar que el cargo corresponda al sector administracion
drop trigger if exists verificar_sector_administracion;
DELIMITER //
create trigger verificar_sector_administracion
before insert on empleados
for each row
begin
if new.Cargo != "Administrativo" and new.Cargo != "Gerente" and new.Cargo != "Director" and new.Sector = "Administracion" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector atencion al cliente
drop trigger if exists verificar_sector_aac;
DELIMITER //
create trigger verificar_sector_aac
before insert on empleados
for each row
begin
if new.Cargo != "Comunicador" and new.Cargo != "Registrador de reclamos" and new.Sector = "Atencion al cliente" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

```
-- trigger empleados: verificar que el cargo corresponda al sector finanzas
drop trigger if exists verificar_sector_finanzas;
DELIMITER //
create trigger verificar_sector_finanzas
before insert on empleados
for each row
begin
if new.Cargo != "Contador" and new.Cargo != "Administrador de ingresos" and new.Sector = "Finanzas" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector mantenimiento
drop trigger if exists verificar_sector_mantenimiento;
DELIMITER //
create trigger verificar_sector_mantenimiento
before insert on empleados
for each row
begin
if new.Cargo != "Mecanico" and new.Cargo != "Tecnico" and new.Sector = "Mantenimiento" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

Triggers vehículos:

```sql
-- trigger vehiculos: evitar registrar vehiculo con precio negativo
drop trigger if exists precio_negativo;
DELIMITER //
create trigger precio_negativo
before insert on vehiculos
for each row
begin
if new.precio < 0 then
signal sqlstate '45000'
set message_text = "Error. El precio no puede ser negativo";
end if;
end //
DELIMITER ;
```

```sql
-- trigger vehiculos: evitar registrar vehiculo con año imposible
drop trigger if exists año_imposible;
DELIMITER //
create trigger año_imposible
before insert on vehiculos
for each row
begin
if new.anio < 2006 or new.anio > year(curdate()) then
signal sqlstate '45000'
set message_text = "Error. El año de fabricacion no es valido";
end if;
end //
DELIMITER ;
```

```sql
-- trigger vehiculos: impide insertar un vehículo si el empleado asignado no existe.
drop trigger if exists chequear_empleado;
DELIMITER //
create trigger chequear_empleado
before insert on vehiculos
for each row
begin
declare existe_empleado int default null;
if new.ID_Empleado is not null then
    select ID_Empleado into existe_empleado from Empleados
    where ID_Empleado = new.ID_Empleado;

    if existe_empleado is null then
    signal sqlstate '45000'
    set message_text = "Error. El empleado asignado al vehiculo no existe.";
    end if;
end if;
end //
DELIMITER ;
```

```sql
-- trigger vehiculos: impide insertar un vehiculo si el empleado asignado no pertenece al sector "Transporte".
drop trigger if exists chequear_sector;
DELIMITER //
create trigger chequear_sector
before insert on vehiculos
for each row
begin
declare existe_empleado int default null;
declare sector_emp varchar(100);
if new.ID_Empleado is not null then
    select sector into sector_emp from Empleados
    where ID_Empleado = new.ID_Empleado;

    if sector_emp != "Transporte" then
    signal sqlstate '45000'
    set message_text = "Error. El empleado asignado no corresponde al sector 'Transporte'.";
    end if;
end if;
end //
DELIMITER ;
```

```sql
-- Trigger vehículos: impide eliminar un vehiculo que tenga empleados asociados
drop trigger if exists limitar_eliminar_vehiculo;
DELIMITER //
create trigger limitar_eliminar_vehiculo
before delete on Vehiculos
for each row
begin
if old.ID_Empleado is not null then
signal sqlstate '45000'
set message_text = "Error. No se puede eliminar el vehiculo ya que tiene empleados asociados.";
end if ;
end //
DELIMITER ;
```

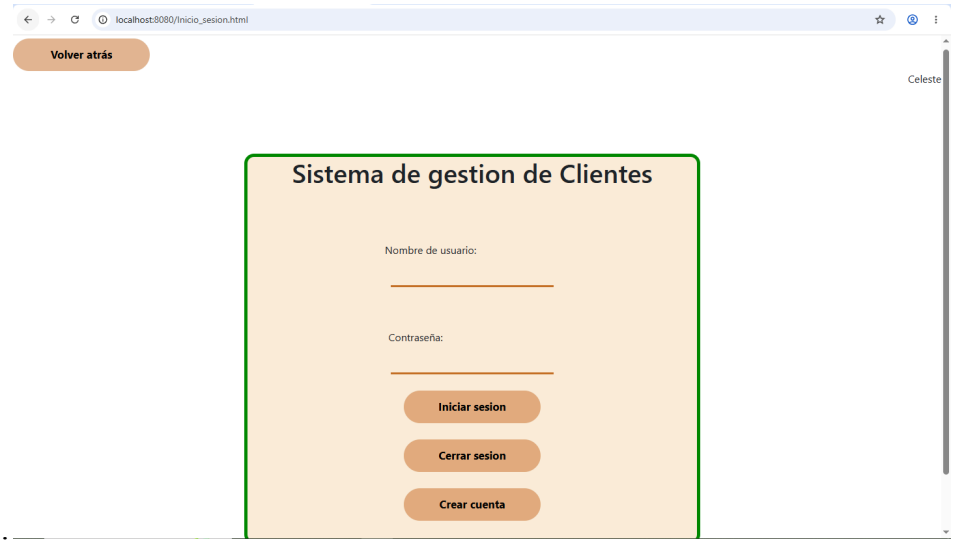
Hubo un detalle interesante sobre uno de los triggers de vehículos, más precisamente el trigger que impide ingresar vehículos creados en un “año imposible”.
Cuando la Inteligencia Artificial me dio la idea, solo me decía que con año imposible se refería a años posteriores al año actual 2026, pero yo me di cuenta que tambien deberia haber un año maximo de antiguedad en los vehículos de una empresa logística, tanto por temas de seguridad como de modernidad. Por lo que decidí investigarlo, y lo pensé correctamente, ya que según la Ley Nacional de Tránsito N° 24.449 (artículo 53) y su decreto reglamentario N° 779/95, los transportes de cargas generales en el país no pueden superar los 20 años de antigüedad.
Tras conocer acerca de esto, en el trigger puse como año mínimo de fabricación el año 2006.

El dia **30/10**, durante las horas de análisis de sistemas, le consulté a mi compañero Saragusti, quien realizo la mayoria del codigo correspondiente a la conexión de JAVA con el servidor HTTP para que me explique a detalle cómo funcionaba cómo funcionaba el mismo, ya que es en lo que a mi me falta trabajar y que más se me complica comprender.
Me explico el funcionamiento de cada bloque de código, si bien el concepto lo entendí, voy a estudiarlo a detalle para el día de la defensa individual.
Además, aproveché el momento para hablar con él sobre cuestiones de organización del grupo y el reparto de tareas a lo largo del proyecto, de igual manera lo consulte con el líder Goldfarb.

El día **2/11** realice los diseños CSS correspondientes a las páginas de iniciar sesión y crear usuario.
Por el lado de Iniciar Sesión, primero modifiqué el HTML debido a que el section que incluye el código para ingresar los datos, estaba como campos input. Para mayor comodidad visual y dado

que ya había aprendido a implementarlos en el proyecto aparte de Tuinyla, lo cambié a un <form>. Posteriormente, modifique el JavaScript para adaptarlo.

Luego, para el CSS, me inspiré en un formulario similar que había realizado para otro proyecto, adaptando los datos a ingresar a los requeridos. También modifique el aspecto de todos los botones. El resultado final es el siguiente:



Luego para la parte de Crear usuario, a la cual se accede con el tercer botón del formulario, tuve que configurar prácticamente todo el HTML, JavaScript y CSS de cero (de por si CSS no tenia). El codigo HTML + JavaScript solo constaba de una plantilla primitiva, no funcional y con datos faltantes. Aprovechando la plantilla y con algunas ayudas de Saragusti, deje el codigo asi:

```html
<!DOCTYPE html>
<html lang = "en">
<head>
    <meta charset = "UTF-8">
    <meta name = "viewport" content = "width=device-width, initial-scale=1.0">
    <title>Crear cuenta</title>
   <link rel="stylesheet" href="Crear.css">
</head>
<body>
    <button class="atras" onclick="history.back()">Volver atrás</button>
    <section class="centrar">
        <form class="crearCuenta">
            <h1>Crear cuenta de cliente: </h1>
            <label class="textoUsuario" for="Usuario">Nombre:</label>
            <input id="Usuario"  type="text" placeholder="Nombre">
            <label class="textoApellido" for="apellido">Apellido:</label>
            <input id = "apellido" type="text" placeholder="Apellido">
            <label class="textoDNI" for="dni">DNI:</label>
            <input id="dni" type="text" placeholder="DNI">
            <label class="textoDireccion" for="dire">Direccion:</label>
            <input id="dire" type="text" placeholder="Direccion">
            <label class="textoTelefono" for="tel">Telefono:</label>
            <input id="tel" type="text" placeholder="Telefono">
            <label class="textoEmail" for="email">E-Mail:</label>
            <input id="email" type="text" placeholder="E-Mail">
            <label class="textoCodigoP" for="cp">Codigo Postal:</label>
            <input id="cp" type="text" placeholder="Codigo Postal">
            <label class="textoContraseña" for="Contra">Contraseña:</label>
            <input id="Contra" type="text" placeholder="Contraseña">
            <button class="agregar" onclick="agregar_cuenta()">Crear Cuenta</button>
        </form>
    </section>
```

HTML

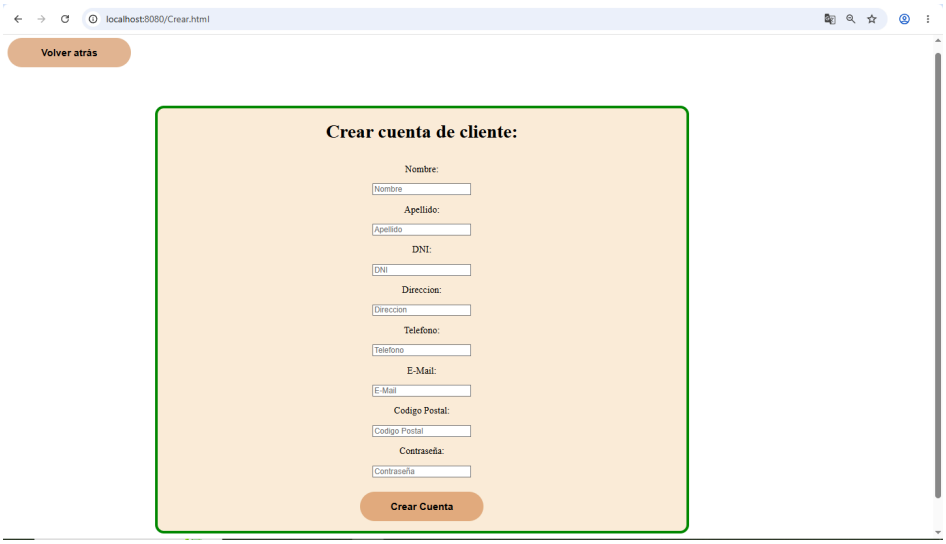
```javascript
    <script>
        var listaa=JSON.parse(localStorage.getItem("lista"));
        function agregar_cuenta (){
            let ids=document.getElementById("Usuario").value;
            let apellido= document.getElementById("apellido").value;
            let dni= document.getElementById("dni").value;
            let dire= document.getElementById("dire").value;
            let tel= document.getElementById("tel").value;
            let email= document.getElementById("email").value;
            let cp= document.getElementById("cp").value;
            let contras=document.getElementById("Contra").value;

            let auxiliar = {
            Nombre: ids,
            Apellido: apellido,
            DNI: dni,
            Direccion: dire,
            telefono: tel,
            email: email,
            Codigo_postal: cp,
            Contrasenia: contras
            };
            listaa.push(auxiliar);
            window.open('Inicio_sesion.html', '_self');
            localStorage.setItem("lista", JSON.stringify(listaa));
        }

    </script>
</body>
```

JavaScript (por medio de <script>)

Y sumandole el CSS, el cual tiene la misma plantilla que el utilizado para iniciar sesión, el resultado final de Crear cuenta es el siguiente:



El día martes **3/11**, en mi casa, retome con los códigos que tenía pendientes desde mi semana de inactividad (10/10 - 19/10 aprox.).

Hasta el momento, solo tenía realizada una primera versión del código de Java para agregar empleados (del servidor web). Este día, finalicé del todo el código para agregar empleados, comencé y complete los de editar empleado y eliminar empleado, todo a partir de plantillas ya realizadas dentro de la clase, adaptándola a lo solicitado.

Como en mi casa tengo una version de NetBeans distinta a la usada en las PCs del colegio, las conexiones entre Java, HTTPS y la base de datos no me andaban correctamente, por lo que la unica manera en el momento que tenia para verificar que esten bien los codigos fue preguntarselo a la Inteligencia Artificial (ChatGPT). Le pase toda la clase completa y le pregunte si, a partir de ello, mis codigos estaban bien. Tenian la estructura correcta, solo me habia equivocado en un par de detalles menores, que en 10 segundos los corregi.

Los errores fueron:

En Agregar Empleado:

sentencia.setString(6, Sector);

sentencia.setString(7, Cargo);

sentencia.setString(8, Rol);

sentencia.registerOutParameter(7, java.sql.Types.VARCHAR); ←— ACÁ IBA 9, NO 7

En editar empleado:

String sql = "UPDATE empleados SET nombre = ?, apellido = ?, "

+ "dni = ?, email = ?, telefono = ?, sector = ?" ←— ME FALTO UNA COMA

+ "cargo = ?, rol = ? WHERE ID_Empleado = ?";

Los códigos finales, quedaron de la siguiente manera:

CREAR EMPLEADO (agregar empleado):

```
});
servidor.createContext("/api/Empleados/crear", new HttpHandler() {

    @Override
    public void handle(HttpExchange intercambio) throws IOException {

        if (!intercambio.getRequestMethod().equals("POST")) {
            enviarRespuesta(intercambio, 405,
                    "{\"error\":\"Metodo no permitido\"}");
            return;
        }

        // Leer los datos que llegan desde el formulario.
        java.io.InputStream entrada = intercambio.getRequestBody();
        java.io.ByteArrayOutputStream buffer =
                new java.io.ByteArrayOutputStream();

        int dato;
        while ((dato = entrada.read()) != -1) {
            buffer.write(dato);
        }

        String cuerpo = new String(
                buffer.toByteArray(),
                StandardCharsets.UTF_8
        );

        // Separar los datos recibidos.
        java.util.Map<String, String> datos =
                new java.util.HashMap<String, String>();

        String[] parametros = cuerpo.split("&");

        for (String parametro : parametros) {
            String[] partes = parametro.split("=", 2);

            String clave = java.net.URLDecoder.decode(
                    partes[0], "UTF-8"
            );

            String valor = "";
            if (partes.length > 1) {
                valor = java.net.URLDecoder.decode(
                        partes[1], "UTF-8"
                );
            }

            datos.put(clave, valor);
        }

        // Obtener los valores del formulario.
        String Nombre = datos.get("Nombre");
        String Apellido = datos.get("Apellido");
        String Dni = datos.get("Dni");
        String Email = datos.get("email");
        String Telefono = datos.get("telefono");
        String Sector = datos.get("sector");
        String Cargo = datos.get("cargo");
        String Rol = datos.get("rol");
        // Validar los campos.
        if (Nombre == null
                || Apellido == null
                || Dni == null
                || Email == null
                || Telefono == null
                || Sector == null
                || Cargo == null
                || Rol == null
                || Nombre.trim().isEmpty()
                || Apellido.trim().isEmpty()
                || Dni.trim().isEmpty()
                || Email.trim().isEmpty()
                || Telefono.trim().isEmpty()
                || Sector.trim().isEmpty()
                || Cargo.trim().isEmpty()
                || Rol.trim().isEmpty()
                ) {

            enviarRespuesta(intercambio, 400,
                    "{\"error\":\"Todos los campos son obligatorios\"}");
            return;
        }
```

```
                int dni;
                int telefono;

                try {
                    dni = Integer.parseInt(Dni);
                    telefono = Integer.parseInt(Telefono);

                } catch (NumberFormatException e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"El DNI y telefono deben ser numeros enteros\"}");
                    return;
                }

                // Preparar la consulta SQL.
                String sql = "CALL agregar_empleados(?, ?, ?, ?, ?, ?, ?, ?, ?)"; // Mantiene los 7 '?'

try (
    Connection conexion = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/logistica32",
            "root",
            ""
    );
     CallableStatement sentencia = conexion.prepareCall(sql);
) {
    sentencia.setString(1, Nombre);
    sentencia.setString(2, Apellido);
    sentencia.setInt(3, dni);
    sentencia.setString(4, Email);
    sentencia.setInt(5, telefono);
    sentencia.setString(6, Sector);
    sentencia.setString(7, Cargo);
    sentencia.setString(8, Rol);

    // 1. Corregido: Registrar el séptimo parámetro como SALIDA (Ejemplo si devuelve Texto)
    sentencia.registerOutParameter(9, java.sql.Types.VARCHAR);

    sentencia.execute();

    // 2. Opcional: Puedes leer lo que te devolvió la base de datos
    String respuestaBD = sentencia.getString(9);

    enviarRespuesta(intercambio, 201,
            "{\"mensaje\":\"Empleado guardado correctamente\", \"detalle\":\"" + respuestaBD + "\"}");
```

```
} catch (SQLException e) {
    e.printStackTrace();
    enviarRespuesta(intercambio, 500,
            "{\"error\":\"Error al guardar en la base de datos\"}");
}


            }
        });

// --------------------------------------------------------------------------------
```

EDITAR EMPLEADO:

```
servidor.createContext("/api/Empleados/editar", new HttpHandler() {
1997
1998            @Override
                public void handle(HttpExchange intercambio) throws IOException {
2000
2001            if (!intercambio.getRequestMethod().equals("POST")) {
2002                enviarRespuesta(intercambio, 405,
2003                        "{\"error\":\"Metodo no permitido\"}");
2004                return;
2005            }
2006
2007            java.util.Map<String, String> datos = leerParametros(intercambio);
2008
2009            try {
2010                int id = Integer.parseInt(datos.get("id"));
2011                String nombre = datos.get("nombre");
2012                String apellido = datos.get("apellido");
2013                int dni = Integer.parseInt(datos.get("Dni"));
2014                String email = datos.get("email");
2015                int telefono = Integer.parseInt(datos.get("telefono"));
2016                String sector = datos.get("sector");
2017                String cargo = datos.get("cargo");
2018                String rol = datos.get("rol");
2019
2020
2021                if (dni <= 0 || telefono <= 0) {
2022
2023                    enviarRespuesta(intercambio, 400,
2024                            "{\"error\":\"Revisa los datos ingresados\"}");
2025                    return;
2026                }
2027
2028                String sql = "UPDATE empleados SET nombre = ?, apellido = ?, " + "dni = ?, email = ?, telefono = ?, sector = ?," + "cargo = ?, rol = ? WHERE ID_Empleado = ?";
2029
2030                try (
2031                    Connection conexion = DriverManager.getConnection(
2032                            "jdbc:mysql://localhost:3306/logistica32", "root", "");
2033                    PreparedStatement sentencia = conexion.prepareStatement(sql)
2034                ) {
2035
2036                    sentencia.setString(1, nombre);
2037                    sentencia.setString(2, apellido);
2038                    sentencia.setInt(3, dni);
```

```
2038                    sentencia.setInt(3, dni);
2039                    sentencia.setString(4, email);
2040                    sentencia.setInt(5, telefono);
2041                    sentencia.setString(6, sector);
2042                    sentencia.setString(7, cargo);
2043                    sentencia.setString(8, rol);
2044                    sentencia.setInt(9, id);
2045
2046                    int filas = sentencia.executeUpdate();
2047
2048                    if (filas > 0) {
2049                        enviarRespuesta(intercambio, 200,
2050                                "{\"mensaje\":\"Empleado actualizado correctamente\"}");
2051                    } else {
2052                        enviarRespuesta(intercambio, 404,
2053                                "{\"error\":\"No se encontro al empleado\"}");
2054                    }
2055                }
2056
2057            } catch (NumberFormatException e) {
2058                enviarRespuesta(intercambio, 400,
2059                        "{\"error\":\"El DNI y el Telefono deben ser numeros validos\"}");
2060            } catch (SQLException e) {
2062                e.printStackTrace();
2063                enviarRespuesta(intercambio, 500,
                            "{\"error\":\"Error al actualizar la base de datos\"}");
2065            } catch (Exception e) {
2066                enviarRespuesta(intercambio, 400,
                            "{\"error\":\"Faltan datos o son invalidos\"}");
2067            }
2068        }
2069    });
```

ELIMINAR EMPLEADO:

```
servidor.createContext("/api/Empleados/eliminar", new HttpHandler() {

            @Override
            public void handle(HttpExchange intercambio) throws IOException {

                if (!intercambio.getRequestMethod().equals("POST")) {
                    enviarRespuesta(intercambio, 405,
                            "{\"error\":\"Metodo no permitido\"}");
                    return;
                }

                java.util.Map<String, String> datos = leerParametros(intercambio);

                try {
                    int id = Integer.parseInt(datos.get("id"));

                    if (id <= 0) {
                        enviarRespuesta(intercambio, 400,
                                "{\"error\":\"ID no valido\"}");
                        return;
                    }

                    String sql = "DELETE FROM empleados WHERE ID_Empleado = ?";

                    try (
                        Connection conexion = DriverManager.getConnection(
                                "jdbc:mysql://localhost:3306/logistica32", "root", "");
                        PreparedStatement sentencia = conexion.prepareStatement(sql)
                    ) {
                        sentencia.setInt(1, id);

                        int filas = sentencia.executeUpdate();

                        if (filas > 0) {
                            enviarRespuesta(intercambio, 200,
                                    "{\"mensaje\":\"Empleado eliminado correctamente\"}");
                        } else {
                            enviarRespuesta(intercambio, 404,
                                    "{\"error\":\"No se encontro el empleado.\"}");
                        }
                    }
```

```
                } catch (NumberFormatException e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"El ID debe ser numerico\"}");
                } catch (SQLException e) {
                    e.printStackTrace();
                    enviarRespuesta(intercambio, 500,
                            "{\"error\":\"Error al eliminar de la base de datos\"}");
                } catch (Exception e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"No se pudo procesar la solicitud\"}");
                }
            }
        });
```

Lamentablemente, todavía no entiendo el funcionamiento de estos códigos más que por deducción, ya que solo me base en las plantillas realizadas previamente. Por lo que tendré que estudiar bien sus funcionamientos para el día de las defensas personales.

Al día siguiente, el miércoles **4/11** y durante las horas de Redes y Proyecto Informático 2, mi compañero Benjamin Saragusti y yo corregimos juntos los códigos del servidor web en Java que realicé el día anterior.
Una vez que los corroboramos bien, me puse a hacer los códigos de mismo fin, pero correspondientes a la página en HTML.
De igual manera que con los códigos de Java, me base en plantillas reutilizadas por otros integrantes, modificándolas y adaptándolas para que funcionen con empleados.

No voy a agregar capturas de los códigos, ya que son los mismos que quedaron en la versión final de la página, y se pueden encontrar en el zip de la misma. Esto con el fin de no irradiar la carpeta de muchas capturas.
Lo mismo con el funcionamiento, si bien esto fue porque no saque capturas en el momento y, como mencione anteriormente, en mi PC se me dificulta probar las conexiones por el tema de versiones, no subire tampoco porque se vera despues como funciona a la hora de presentar el proyecto y utilizar la página en cualquier momento.

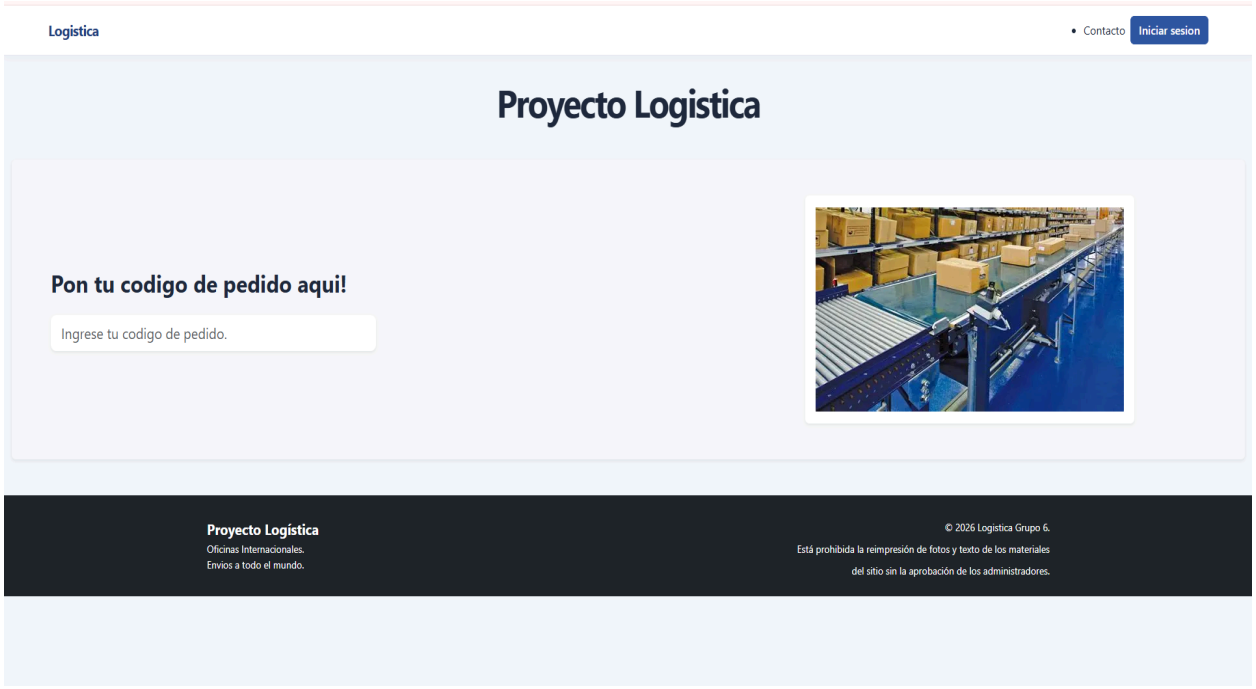
Los probamos en la página y funcionaban perfectamente, los empleados creados se guardaban en la base de datos y se visibilizaban en las tablas de la página, así como también los podías editar y eliminar. Lamentablemente, cree los códigos html de agregar, editar y eliminar empleado y funcionan

El día viernes **6/11** realicé la última actividad que me asignaron referente a lo que es códigos: modificar el aspecto del Index (que incluía header, body y footer) y el footer del Index de admin.

Por el lado del header del index normal, copie el formato del index de admin. Las cosas distintas que tenia este nuevo header eran un efecto hover en el texto “Logistica” (el cual refresca la pagina), y lo mismo con el boton de tu usuario.

En el body, adapte el titulo “Proyecto Logistica” como en el index de admin, y en el footer modifique los textos de copyright, ya que la version anterior contenia unos textos muy chistosos, y elimine los botones para ver envios y paquetes, por orden del lider Goldfarb.

Resultado final del index normal:



También había modificado el footer del index de admin, me lo pidió mi compañero Saragusti para corroborarlo y subirlo él, pero no aparece el cambio. Me dijo que ya no se puede cambiar porque ya se plasmaron en el Cisco.

Osea que en la presentación, tendremos una situación muy ‘’chistosa’’ cuando los profesores vean la seriedad del texto que se encuentra en el footer del index de admins.